

AAED-Exámenes-Estructura-de-Dato...



KukiMW



Análisis de Algoritmos y Estructuras de Datos



2º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



AQUARIUS

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR, PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA... **ESO, SE MERECE UN AQUARIUS.**

SUDAR ES BELLO

AQUARIUS es una marca registrada de The Coca-Cola Company.

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



ANALISIS DE ALGORITMOS Y ESTRUCTURA DE DATOS

Exámenes

Notas del autor	2
Cubilete	3
Tren	6
Cursor	8
Texto	10
Símbolo	12
Cocina	16
Consultorio	18
Hospital	21
Tres en Raya	27
Hipermercado/Linea de Cajas.....	32
Gestor de Impresión	35
Diccionario	39
Token Bus	42
Postfijo	46
Cartelera	48
Escalonada	51

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Notas del autor

No puedo asegurar que los ejercicios estén al 100% correctos o que sean la forma correcta que se busque el día del examen. Son resultado de rehacerlos varias veces tanto por cuenta propia como por recopilación de otros ejercicios y exámenes resueltos. Siempre es aconsejable utilizar solo el enunciado y pensar por uno mismo el proceso, pues es la forma más realista. Se da el ejercicio "hecho" como guía. Algunos vienen con batería de prueba, otros con la especificación.

AQUARIUS®

SABEMOS QUE PUEDES DESCARGARTE
UN TRABAJO COMPLETO DE CUALQUIER
ESCRITOR, PERO ZAMBULLIRTE EN
SU OBRA Y NO PARAR DE LEER
HASTA QUE SE TE HAGA DE DÍA...
ESO, SE MERECE UN AQUARIUS.

SUDAR ES BELLO

AQUARIUS es una marca registrada de The Coca-Cola Company.



Análisis de Algoritmos y Est...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

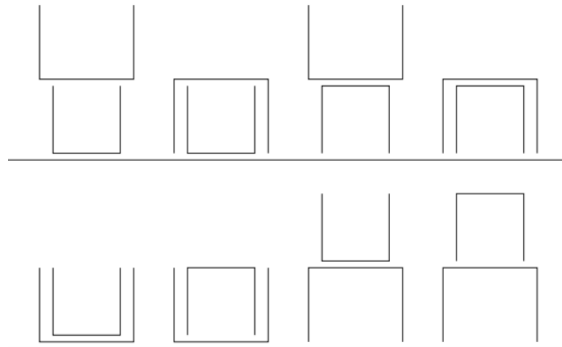
- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR

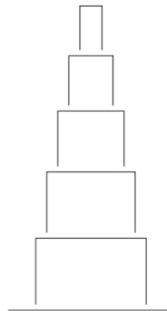


Cubilete

Un conocido juguete está formado por una colección de cubiletes de diferentes tamaños que se pueden apilar colocando cada uno de ellos boca arriba o boca abajo. Así, dos cubos de distinto tamaño se pueden apilar de 8 formas diferentes representadas en la siguiente figura.



Partiendo de una pila de cubiletes, todos de diferente tamaño y en cualquier orden, en la que cada uno de ellos puede estar boca arriba o boca abajo, el juego consiste en construir una nueva pila con todos los cubos boca abajo y ordenados de mayor a menor tamaño (en sentido de abajo hacia arriba).



1 Pila Final 5 cubos

- Defina el tipo de datos `tCubo` para representar los cubiletes del juego.
- Implemente una función que dada una pila cualquiera de cubiletes simule el juego y devuelva la nueva pila construida. La solución se basará exclusivamente en el TAD Pila.

NOTA: Es absolutamente necesario definir todos los tipos implicados en la resolución del problema, así como los prototipos de las operaciones utilizadas de los TADs conocidos.

```
#ifndef CUBILETE_HPP
#define CUBILETE_HPP
#include "../TADS Pila/pilaenla.hpp"
#include <iostream>
#include <cassert>
using namespace std

//a) Defina el tipo de datos tCubo para representar los cubiletes del juego.

struct tCubo{
    int tama;
    bool abajo; //Verdadero si mira hacia abajo, Falso si mira hacia arriba.
    tCubo(int t = 0, bool a = true): tama(t), abajo(a) {} //Constructor
};

//b) Implemente una función que dada una pila cualquiera de cubiletes simule el juego y
// devuelva la nueva pila construida. La solución se basará exclusivamente en el TAD Pila.

void mostrarTorre(Pila<tCubo>& inicio)
{
    Pila<tCubo> copia = inicio;
    Pila<tCubo> inv;
    std::cout << "La torre: ";
    while(!copia.vacia())
    {
        inv.push(copia.tope());
        copia.pop();
    }
    while(!inv.vacia())
    {
        std::cout << inv.tope().tama;
        if(inv.tope().abajo) std::cout << "T ";
        else std::cout << "F ";
        inv.pop();
    }
    std::cout << endl;
}

Pila<tCubo> juego(Pila<tCubo>& inicio)
{
    Pila<tCubo> Final;
    tCubo cubilete;

    std::cout << "Que comience el juego!" << std::endl;
    while (!inicio.vacia())
    {
        cubilete = inicio.tope();
        inicio.pop();

        if(!cubilete.abajo)
            cubilete.abajo = true;

        while (!Final.vacia() && Final.tope().tama < cubilete.tama)
        {
            inicio.push(Final.tope());
            Final.pop();
        }

        Final.push(cubilete);
    }
    std::cout << endl;
    std::cout << "Lo hemos logrado!!" << std::endl;
    return T1;
}

#endif //CUBILETE_HPP
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...
ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
/* NOTA: Es absolutamente necesario definir todos los tipos implicados en la resolución del problema, así como los  
prototipos de las operaciones utilizadas de los TADs conocidos.*/
```

```
/******
```

```
Pila vectorial: secuencia de elementos de un tipo determinado, en  
la cual se pueden añadir y eliminar elementos sólo por uno de sus  
extremos llamado tope o cima.
```

```
privado:
```

```
tElemento *elementos; // Vector de elementos  
unsigned lmax; // Tamaño del vector  
unsigned n_eltos; // Tamaño de la pila
```

```
}
```

```
Prototipos:
```

```
Pila(): crea una pila vacía
```

```
bool vacia(): True si la pila está vacía.
```

```
void pop(): Saca el elemento en el tope de la pila.
```

```
void push(tCubo& x): Inserta el elemento x en el tope de la pila
```

```
const tCubo& tope() const: Devuelve el elemento en el tope de la pila.
```

```
*****/
```

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

```
#include "Cubilete.hpp"
```

```
int main()
```

```
{
```

```
Pila<tCubo> Torre;  
tCubo C1(1,false),C2(2,false),C3(3,true),C4(4,false),C5(5,false);
```

```
Torre.push(C5);Torre.push(C4);Torre.push(C2);Torre.push(C1);Torre.push(C3);  
mostrarTorre(Torre);
```

```
Pila<tCubo> Apilada = juego(Torre);  
mostrarTorre(Apilada);
```

```
return 0;
```

```
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Tren

Realizar la especificación e implementación de un TAD Tren que sirva para representar secuencias (de longitud arbitraria) de vagones (se supone que el tipo vagón está predefinido). Si la secuencia no está vacía, hay un vagón del tren denominado "activo" sobre el que puede actuar un robot manipulador. Las operaciones que se piden son:

- Crear un tren.
- Desplazar a la izquierda: Mueve un tren una posición a la izquierda de manera que el nuevo vagón activo es el que está inmediatamente a la derecha del actual. Si no hay vagón a la derecha no se hace nada.
- Desplazar a la derecha: Simétrica a la anterior.
- Eliminar el vagón activo: Se suprime el vagón activo y se convierte en activo el que está inmediatamente a su derecha. Si éste no existe, se activa el de su izquierda. Y si éste último tampoco existe, el tren queda vacío.
- Insertar vagón activo: Añade un nuevo vagón al tren desplazando el actual Vagón activo y los siguientes una posición hacia la derecha
- Observar el vagón activo: Una cámara fija situada sobre el robot devuelve el vagón activo.
- Tren vacío: indica si el tren tiene vagones.

Nota: Se prohíbe expresamente utilizar una estructura doblemente enlazada. Y se exige que el coste de todas las operaciones sea constante, es decir, independiente del número de vagones del tren. Se valorará la claridad de la especificación y la eficiencia de las operaciones.

```
#ifndef TREN_HPP
#define TREN_HPP

#include "../TADS/pilaenla.hpp"
#include <iostream>
#include <cassert>

using namespace std;

class Tren{
public:
    typedef int vagon;
    Tren(); //Construir un tren vacio
    void desplazarIz(); //Desplazar tren a la izquierda
    void desplazarDer(); //Desplazar tren a la derecha
    void insertar(vagon x); //Insertar vagon como activo. El resto a la derecha.
    void eliminarVagon(); //Eliminar vagon activo
    const vagon observarVagon(); //Comprobar si el tren esta vacio
    bool trenVacio();
private:
    Pila<vagon> IZQ;
    Pila<vagon> DER;
    //vagon vagonactivo;
    //El vagon activo se considera el tope de la pila IZQ.
}

Tren::Tren(){} //vagonactivo = IZQ.tope();

void Tren::desplazarIz(){
    assert(!trenVacio()) //Al menos hay vagón(es) en una pila
    if(!DER.vacio())
    {
        IZQ.push(DER.tope());
        DER.pop();
        //vagonactivo = IZQ.tope();
    }
    else
        cout << "No hay vagon que mover" << endl;
}
```

```
void Tren::desplazarDer()
{
    assert(!trenVacio())
    if(!IZQ.vacio())
    {
        DER.push(IZQ.tope());
        IZQ.pop();
    }
    if(IZQ.vacio()) //Al ser el vagón activo no puede dejarse vacío.
    {
        IZQ.push(DER.tope());
        DER.pop();
    }
    //vagonactivo = IZQ.tope();
}

void Tren::eliminarVagon()
{
    assert(!trenVacio());

    if(!IZQ.vacio()) //Elimina vagón activo
    {
        IZQ.pop();
    }

    if(!DER.vacio()) //Se convierte activo el de su derecha
    {
        IZQ.push(DER.tope());
        DER.pop();
    }
    //vagonactivo = IZQ.tope();

    //Si no hay vagones a la derecha, se vuelve activo el tope de la izquierda.
    //Si no hay vagones a la izquierda, el tren está vacío.
}

void Tren::insertarvagon(const vagon& x)
{
    if(!IZQ.vacio())
    {
        DER.push(IZQ.tope());
        IZQ.pop();
    }
    IZQ.push(x);
    //vagonactivo = IZQ.tope();
}

const vagon Tren::observarVagon()
{
    assert(!trenVacio())
    return IZQ.tope();
    //return vagonactivo
}

bool Tren::trenVacio() noexcept
{ return IZQ.vacio() && DER.vacio(); } //Si no hay vagon en ninguna pila.

#endif // TREN_HPP
```

Cursor

En las operaciones de entrada y salida de datos es muy frecuente que los programas ofrezcan al usuario la posibilidad de editar los datos. Esto ocurre especialmente cuando los datos consisten en una línea de texto. En este caso el usuario puede introducir y modificar los caracteres en modo interactivo pulsando teclas que le permiten mover el cursor por la línea e insertar y suprimir caracteres. El modo de operación habitual es el siguiente:

- El cursor se puede situar en cualquier posición desde el primer carácter de la línea hasta la posición siguiente al último carácter de la línea.
 - Cada vez que se introduce un carácter, el cursor avanza a la siguiente posición a la derecha.
 - En modo inserción los caracteres que están en la posición del cursor y siguientes se desplazarán para dejar sitio al nuevo.
 - En modo sobreescritura se sustituye el carácter de la posición del cursor por el nuevo.
 - En ambos modos de escritura, si el cursor está en la última posición de la línea, simplemente se añaden al texto los caracteres introducidos y el cursor continúa en la última posición.
- a) Defina una estructura de datos basada en el TAD Pila para representar una línea de texto de cualquier longitud.
- b) Escriba funciones que realicen con la estructura definida las siguientes operaciones:
- o avanzar el cursor una posición
 - o borrar el carácter de la posición del cursor
 - o retrasar el cursor una posición
 - o borrar el carácter anterior al cursor
 - o ir al final de la línea
 - o insertar un carácter en la posición del cursor
 - o ir al principio de la línea
 - o sobreescribir el carácter del cursor

```
#ifndef CURSOR2023_HPP
#define CURSOR2023_HPP

#include <iostream>
#include <cassert>
#include "pilaenla.hpp"

template <typename T>
class Cursor{
public:
    Cursor(){};
    void Avanzar();
    void Retrasar();
    void Final();
    void Principio();
    void Suprimir();
    void Borrar();
    void Insertar(const T& x);
    void Sobreescribir(const T& x);
    ~Cursor();

private:
    Pila<char> D;
    Pila<char> I;
};
```

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
inline void Cursor::Avanzar()
{
    assert(!D.vacia());
    I.push(D.tope());
    D.pop;
}

inline void Cursor::Retrasar()
{
    assert(!I.vacia());
    D.push(I.tope());
    I.pop;
}

void Cursor::Final()
{
    while(!D.vacia())
    {
        I.push(D.tope());
        D.pop;
    }
}

void Cursor::Principio()
{
    while(!I.vacia())
    {
        D.push(I.tope());
        I.pop;
    }
}

inline void Cursor::Suprimir()
{
    assert(!D.vacia());
    D.pop();
}

inline void Cursor::Borrar()
{
    assert(!I.vacia())
    I.pop();
}

inline void Cursor::Insertar(const T& x)
{
    I.push(x);
}

inline void Cursor::Sobreescribir(const T& x)
{
    assert(!I.vacia());
    I.pop();
    I.push(x);
}

Cursor::~~Cursor()
{
    D.~Pila();
    I.~Pila();
}

#endif
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Texto

Diseña la estructura de datos para almacenar en memoria un texto del cual desconocemos el número de líneas y de caracteres por línea. Sabemos que el texto solo consta de caracteres alfanuméricos y signos de puntuación, es decir, en él no aparecen caracteres de control como fin de línea, retorno de carro...

Sin embargo, hay caracteres especiales que pueden aparecer un número indeterminado de veces.

@ indica el borrado del carácter anterior. En caso de que aparezca varios consecutivos, solo tiene efecto el primero; el resto, aunque se almacene, se ignora. Este carácter no aparece al principio de ninguna línea.

indica el borrado de la línea completa. Si aparecen varias igual, solo aparecerá al final de la línea.

Escribe una función MostrarTexto().

Al realizar este volcado de pantalla no te preocupes por el número de caracteres ni de líneas.

```
#ifndef TEXTO_HPP
#define TEXTO_HPP
#include "../TADS Lista/listaenla.hpp"
#include <iostream>
using namespace std;

typedef Lista<char> Linea;

class Texto
{
public:
    Texto(){}
    void insertarLinea(Linea l);
    void MostrarTexto();
private:
    Lista<Linea> T; //Lista de lineas, que son listas de caracteres.
};

void Texto::insertarLinea(Linea l)
{
    T.insertar(l,T.fin());
}

void Texto::MostrarTexto()
{
    Linea Lmostrar;
    Lista<Linea>::posicion pT = T.primer(); //Posición de lineas
    Linea::posicion pL; //Posicion de caracteres

    while(pT != T.fin()) //Mientras no acabe el texto
    {
        Lmostrar = T.elemento(pT); //Guardar Linea
        pL = Lmostrar.primer(); //Primer carácter de la linea Lmostrar

        //Si encontramos # como último carácter, eliminamos esa linea
        if((Lmostrar.primer() != Lmostrar.fin()) &&
            (Lmostrar.elemento(Lmostrar.anterior(Lmostrar.fin())) == '#'))
        {
            //T.eliminar(pT);
            while(pL != Lmostrar.fin())
            {
                Lmostrar.eliminar(pL);
            }
        }

        while(pL != Lmostrar.fin()) //Mientras no acabe la Linea
        {
            if(Lmostrar.siguiente(pL) != Lmostrar.fin() && Lmostrar.elemento(Lmostrar.siguiente(pL))=='@')
            {
                Lmostrar.eliminar(pL);
                //Mientras siga siendo el caracter @
                while(pL != Lmostrar.fin() && Lmostrar.elemento(pL)=='@')
                {
                    Lmostrar.eliminar(pL);
                }
            }
        }
    }
}
```

```
        if(pL != Lmostrar.fin())
        {
            cout << Lmostrar.elemento(pL);
            pL = Lmostrar.siguiente(pL);    //Iterar Caracteres
        }
    }
    cout<<endl;
    if((pT != T.fin()))
    {pT = T.siguiente(pT);}                //Iterar Lineas
}

#endif //TEXTO_HPP

////////////////////////////////////

#include "Texto.hpp"
#include <iostream>
using namespace std;

int main()
{
    Texto t;
    Linea L1, L2, L3;

    //Inicializar Lineas
    L1.insertar('b',L1.fin());
    L1.insertar('e',L1.fin());
    L1.insertar('a',L1.fin());
    L1.insertar('t',L1.fin());
    L1.insertar('t',L1.fin());
    L1.insertar('@',L1.fin());
    L1.insertar('r',L1.fin());
    L1.insertar('i',L1.fin());
    L1.insertar('z',L1.fin());

    L2.insertar('i',L2.fin());
    L2.insertar('s',L2.fin());
    L2.insertar('a',L2.fin());
    L2.insertar('@',L2.fin());
    L2.insertar('@',L2.fin());
    L2.insertar('b',L2.fin());
    L2.insertar('e',L2.fin());
    L2.insertar('l',L2.fin());
    L2.insertar('@',L2.fin());

    L3.insertar('b',L3.fin());
    L3.insertar('e',L3.fin());
    L3.insertar('a',L3.fin());
    L3.insertar('t',L3.fin());
    L3.insertar('t',L3.fin());
    L3.insertar('r',L3.fin());
    L3.insertar('i',L3.fin());
    L3.insertar('z',L3.fin());
    L3.insertar('#',L3.fin());

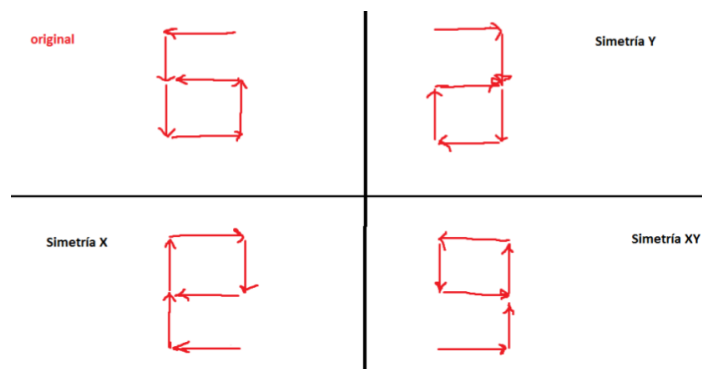
    //Inicializar texto
    t.insertarLinea(L1);t.insertarLinea(L2);t.insertarLinea(L3);
    //Mostrar texto
    t.MostrarTexto();

    return 0;
}
```

Símbolo

El TAD símbolo se usa para representar símbolos trazados con líneas rectas. Un símbolo es una sucesión de Trazos, un trazo es una línea recta con sentido arriba, abajo, izquierda y derecha. El TAD símbolo se tiene que permitir realizar las siguientes operaciones:

- Crear un símbolo vacío.
- Añadir un trazo al final de un símbolo.
- Deshacer los últimos n trazos.
- Devolver símbolo con simetría X (Volteado Vertical)
- Devolver símbolo con simetría Y (Volteado Horizontal)
- Devolver símbolo con simetría XY



- Realice la especificación del TAD
- Diseñe una estructura de datos adecuada para representar el TAD e implemente las operaciones anteriores.

NOTA: Es absolutamente necesario definir todos los tipos de datos implicados en la resolución del ejercicio, así como los prototipos de las operaciones utilizadas de los TADs conocidos.

```
#ifndef SIMBOLO_LISTA_HPP
#define SIMBOLO_LISTA_HPP
#include "../TADS Lista/listaenla.hpp"
#include <iostream>
#include <cassert>
using namespace std;

enum Trazo {A,B,D,I}; //Arriba, Bajar, Derecha, Izquierda

class Simbolo{
public:
    Simbolo();
    void insertarTrazo(Trazo x);
    void eliminarTrazos(int n);
    Simbolo simetriaX();
    Simbolo simetriaY();
    Simbolo simetriaXY();
    void mostrarSimbolo();
private:
    Lista<Trazo> S;
    Simbolo(const Lista<Trazo>& listaTrazos);
};

// <-- CONSTRUCTOR --> //
Simbolo::Simbolo(){}
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
// <-- CONSTRUCTOR PRIVADO --> //
//Crear un objeto Simbolo a partir de una lista de trazos.
Simbolo::Simbolo(const Lista<Trazo>& listaTrazos): S(listaTrazos) {}

// <-- INSERTAR --> //
void Simbolo::insertarTrazo(Trazo x)
{ S.insertar(x, S.primer()); }

// <-- ELIMINAR N TRAZOS --> //
void Simbolo::eliminarTrazos(int n){
    assert(S.primer() != S.fin());
    while(S.primer() != S.fin() && n > 0)
    {
        S.eliminar(S.primer());
        n--;
    }
}

// <-- SIMETRIA X --> //
Simbolo Simbolo::simetriaX(){
    Lista<Trazo> copia = S;
    Lista<Trazo>::posicion p = copia.primer();
    while(p != copia.fin())
    {
        switch(copia.elemento(p))
        {
            case A:
                copia.elemento(p) = B;
                break;
            case B:
                copia.elemento(p) = A;
                break;
        }
        p = copia.siguiete(p);
    }
    return Simbolo(copia);
}

// <-- SIMETRIA Y --> //
Simbolo Simbolo::simetriaY(){
    Lista<Trazo> copia = S;
    Lista<Trazo>::posicion p = copia.primer();
    while(p != copia.fin())
    {
        switch(copia.elemento(p))
        {
            case I:
                copia.elemento(p) = D;
                break;
            case D:
                copia.elemento(p) = I;
                break;
        }
        p = copia.siguiete(p);
    }
    return Simbolo(copia);
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
// <-- SIMETRIA XY --> //
Simbolo Simbolo::simetriaXY(){
    Lista<Trazo> copia = S;
    Lista<Trazo>::posicion p = copia.primer();
    while(p != copia.fin())
    {
        switch(copia.elemento(p))
        {
            case A:
                copia.elemento(p) = B;
                break;
            case B:
                copia.elemento(p) = A;
                break;
            case I:
                copia.elemento(p) = D;
                break;
            case D:
                copia.elemento(p) = I;
                break;
        }
        p = copia.siguiente(p);
    }
    return Simbolo(copia);
}

// <-- OSTREAM --> //
void Simbolo::mostrarSimbolo()
{
    Lista<Trazo>::posicion p = S.primer();
    std::cout << " El simbolo: ";
    while(p != S.fin())
    {
        switch(S.elemento(p))
        {
            case A:
                std::cout << "A ";
                break;
            case B:
                std::cout << "B ";
                break;
            case I:
                std::cout << "I ";
                break;
            case D:
                std::cout << "D ";
                break;
        }
        p = S.siguiente(p);
    }
    std::cout << std::endl;
}

#endif // SIMBOLO_LISTA_HPP
```

```
#include "SimboloLista.hpp"

int main()
{
    Simbolo S1;

    //Simbolo vacio
    cout << "VACIO:" <<endl;
    S1.mostrarSimbolo();
    cout << "INSERTAR:" <<endl;

    //Insertar trazo
    S1.insertarTrazo(A);
    S1.mostrarSimbolo();

    //+= trazos
    S1.insertarTrazo(A);S1.insertarTrazo(A);S1.insertarTrazo(I);S1.insertarTrazo(B);S1.insertarTrazo(B);
    S1.insertarTrazo(D);S1.insertarTrazo(I);S1.insertarTrazo(I);S1.insertarTrazo(A);S1.insertarTrazo(D);
    S1.mostrarSimbolo();

    cout << "SIMETRIA:" <<endl;
    S1.mostrarSimbolo();
    //Simetria X
    S1.simetriaX().mostrarSimbolo();
    //Simetria Y
    S1.simetriaY().mostrarSimbolo();
    //Simetria XY
    S1.simetriaXY().mostrarSimbolo();

    cout << "ELIMINAR:" <<endl;
    //Eliminar 5 trazos
    S1.eliminarTrazos(5);
    S1.mostrarSimbolo();
    //Eliminar 1 trazo
    S1.eliminarTrazos(1);
    S1.mostrarSimbolo();
    //Eliminar 0 trazo
    S1.eliminarTrazos(0);
    S1.mostrarSimbolo();
    //Eliminar 10 trazo
    S1.eliminarTrazos(10);
    S1.mostrarSimbolo();

    return 0;
}
```

Cocina

Una empresa de muebles de cocina necesita un TAD para representar el conjunto de muebles colocados en la pared de una cocina. Una cocina se crea con una longitud positiva, y un mueble colocado en la pared se identifica con el par formado por su posición (distancia desde su lateral izquierda al extremo izquierdo de la pared) y su anchura (la profundidad y altura no tiene interés, pues son iguales para todos los muebles).

El TAD debe soportar las siguientes operaciones:

- Crear una cocina vacía con una longitud dada.
- Determinar si un mueble de una cierta anchura puede colocarse en una posición dada.
- Añadir un mueble de una determinada longitud a una posición dada.
- Devolver el mueble *i*-ésimo de la cocina empezando a contar por la izquierda.
- Eliminar el mueble *i*-ésimo de la cocina, si existe.
- Mover el mueble *i*-ésimo de la cocina (si existe) hacia la izquierda, hasta que se junte con el mueble (*i*-1)-ésimo o el extremo izquierdo de la pared.

Se pide:

- a) Especificar el TAD Cocina
- b) Diseñar una estructura de datos adecuada para representar el TAD Cocina.
- c) Implementar las operaciones anteriormente descritas.

```
#ifndef COCINA_HPP
#define COCINA_HPP
#include "../TADS Lista/listadoble.hpp"
#include <iostream>
#include <cassert>
using namespace std;

struct mueble{
    double anchura,pos;      //POS: Lateral izquierdo del mueble hasta extremo izquierdo de la cocina.
    mueble(double a, double p):pos(p),anchura(a){};
};

class Cocina
{
    typedef Lista<mueble>::posicion posicion_cocina;
public:
    Cocina(double l);
    bool colocable(mueble m);
    void insertarMueble(double a,double p);
    mueble devuelveMueble(double num);
    void eliminaMueble(double num);
    void moverMueble(double num);
    ~Cocina();
private:
    Lista<mueble> C;          //Lista de muebles
    double nMuebles, longitud; //Numero de muebles en la lista, Máximo de la lista
    //typedef Lista<mueble>::posicion posicion_cocina;
};

Cocina::Cocina(double l):nMuebles(0),longitud(l){};
```

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
bool Cocina::colocable(mueble m)
{
    bool valido;

    //Si el mueble es más grande que la cocina
    if(m.anchura + m.pos > longitud) {valido = false;}

    //Si no hay muebles puestos en la cocina
    if(C.primer() != C.fin() && valido)
    {
        typename Lista<mueble>::posicion posicion_cocina;
        for(posicion_cocina it = C.primer(); it != C.fin() && valido ; it = C.siguiente(it))
        {
            //Si su posicion coincide con la de otro mueble
            if(m.pos == C.elemento(it).pos)
            {valido = false;}
            //Si la posicion + anchura coincide con el comienzo del proximo mueble
            else if (m.pos < C.elemento(it).pos &&
                     (m.pos + m.anchura) >= (C.elemento(it).pos))
            {valido = false;}
            //Si la posicion coincide con el mueble anterior
            else if ((m.pos > C.elemento(it).pos) &&
                     (m.pos <= C.elemento(it).pos + C.elemento(it).anchura))
            {valido = false;}
        }
    }

    return valido;
}

void Cocina::insertarMueble(double a,double p)
{
    mueble m(a,p);
    if(colocable(m))
    {
        typename Lista<mueble>::posicion posicion_cocina;
        posicion_cocina it = C.primer();
        //Recorrer la lista hasta llegar a la posicion correspondiente
        while(it != C.fin() && C.elemento(it).pos <= m.pos)
            {it = C.siguiente(it);}

        C.insertar(mueble(a,p),it);
        nMuebles++;
    }
}

mueble Cocina::devuelveMueble(double num)
{
    assert(num >= 0 && num <= nMuebles);

    double i = 1;
    typename Lista<mueble>::posicion posicion_cocina;
    posicion_cocina it = C.primer();

    while(i < num)
    {
        it = C.siguiente(it);
        i++;
    }

    //num = i --> it = num
    return C.elemento(it);
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Consultorio

La dirección de un hospital quiere informatizar su consultorio médico con un programa que realice las siguientes operaciones:

- Generar un consultorio vacío.
- Dar de alta a un médico.
- Dar de baja a un médico.
- Poner a un paciente en la lista de espera de un médico.
- Consultar al paciente a quien le toca el turno para ser atendido.
- Atender al paciente que le toque por parte de un médico.
- Comprobar si un médico determinado tiene o no pacientes en espera.

a) Realice la especificación del TAD

b) Diseñe una estructura de datos adecuada para representar el TAD e implemente las operaciones anteriores.

NOTA: Es absolutamente necesario definir todos los tipos de datos implicados en la resolución del ejercicio, así como los prototipos de las operaciones utilizadas de los TADs conocidos.

a) *Realice la especificación del TAD*

Operaciones TAD Consultorio:

```
Consultorio();  
    Post: Generar un consultorio vacío.  
void AltaMedico(Medico M);  
    Post: Dar de alta a un médico.  
void BajaMedico(Medico M);  
    Post: Dar de baja a un médico. Si no está, no se hace nada.  
bool Comprobar(Medico M);  
    Post: Comprobar si un médico determinado tiene o no pacientes en espera.  
void ListaEspera(Medico M, Paciente P);  
    Pre: Debe existir el médico.  
    Post: Poner a un paciente en la lista de espera de un médico.  
Paciente ConsultarTurno(Medico M);  
    Post: Consultar al paciente a quien le toca el turno para ser atendido.  
    Si el médico no tiene pacientes en cola, devuelve vacío.  
void Atender(Medico M, Paciente P);  
    Post: Atender al paciente que le toque por parte de un médico.  
    Si el médico no tiene pacientes en cola, no hace nada.  
~Consultorio();  
    Post: Destruye el consultorio
```

b) *Diseñe una estructura de datos adecuada para representar el TAD e implemente las operaciones anteriores.*

```
#ifndef CONSULTORIO_HPP  
#define CONSULTORIO_HPP  
#include "../TADS Cola/colaenla.hpp"  
#include "../TADS Lista/listadoble.hpp"  
#include <iostream>  
using namespace std;  
  
struct Paciente  
{  
    int id_p; //ID del paciente  
    Paciente(int id): id_p(id){};  
};  
struct Medico  
{  
    int id_m; //ID del médico  
    Medico(int id): id_m(id){};  
    Cola<Paciente> Pacientes; //Cola vacía al crear  
};
```

```
class Consultorio
{
    public:
        typedef Lista<Medico>::posicion posicion;
        Consultorio();
        void AltaMedico(int M);
        void BajaMedico(int M);
        bool Comprobar(int M);
        void ListaEspera(int M, int P);
        Paciente ConsultarTurno(int M);
        void Atender(int M);
        ~Consultorio();
    //private:
        Lista<Medico> Medicos;
        Lista<Medico>::posicion buscar(int M); //Metodo privados
};

//Método privado: BUSCAR MÉDICO
Lista<Medico>::posicion Consultorio::buscar(int M)
{
    Lista<Medico>::posicion p = Medicos.primer();
    //Mientras no llegue a fin y el id del medico sea distinto al buscado.
    while((p != Medicos.fin()) && (Medicos.elemento(p).id_m != M))
    {
        p = Medicos.siguiete(p);
    }
    return p;
}

// <-- CONSTRUCTOR --> //
Consultorio::Consultorio() {};

// <-- DAR DE ALTA A UN MÉDICO --> //
void Consultorio::AltaMedico(int M)
{
    Medico m(M);
    Medicos.insertar(m, Medicos.fin());
}

// <-- DAR DE BAJA A UN MÉDICO --> //
void Consultorio::BajaMedico(int M)
{
    Medicos.eliminar(Medicos.buscar(M));
}

// <-- COMPROBAR QUE TENGA PACIENTES --> //
bool Consultorio::Comprobar(int M)
{
    return Medicos.elemento(Medicos.buscar(M)).Pacientes.vacia();
    //Compruebla la cola de Pacientes del médico con id M
}

// <-- INSERTAR PACIENTE EN LISTA DE ESPERA --> //
void Consultorio::ListaEspera(int M, int P)
{
    Medicos.elemento(Medicos.buscar(M)).Pacientes.push(Paciente(P));
}

// <-- CONSULTAR PROXIMO PACIENTE --> //
Paciente Consultorio::ConsultarTurno(int M)
{
    return Medicos.elemento(Medicos.buscar(M)).Pacientes.frente();
}

// <-- ATENDER PACIENTE --> //
void Consultorio::Atender(int M)
{
    Medicos.elemento(Medicos.buscar(M)).Pacientes.pop();
}
```

```
// <-- DESTRUCTOR --> //
Consultorio::~Consultorio()
{
    Lista<Medico>::posicion p = Medicos.primer();
    //Destruir todas las colas de pacientes
    while(p != Medicos.fin())
    {
        Medicos.elemento(p).Pacientes.~Cola();
        p = Medicos.siguiete(p);
    }
    //Destruir todos los médicos
    Medicos.~Lista();
}

#endif //CONSULTORIO_HPP

////////////////////////////////////

#include "Consultorio.hpp"

void mostrarMedicos(Consultorio C)
{
    Consultorio::posicion p = C.Medicos.primer();
    cout << "Medicos del consultorio:";
    while(p != C.Medicos.fin())
    {
        cout << " " << C.Medicos.elemento(p).id_m << " ";
    }
    cout << endl;
}

int main()
{
    //Crear médicos y pacientes
    Medico M1,M2,M3;
    Paciente P1,P2,P3,P4,P5;

    //Inicializar médicos y pacientes.
    M1.id_m = 123; M2.id_m = 456; M3.id_m = 678;
    P1.id_p = 1; P2.id_p = 2; P3.id_p = 3; P4.id_p = 4; P5.id_p = 5;

    //Crear Consultorio
    Consultorio C1;

    //Meter a los médicos al consultorio;
    C1.AltaMedico(M1);C1.AltaMedico(M2);C1.AltaMedico(M3);
    mostrarMedicos(C1);

    //Eliminar médico
    C1.BajaMedico(M2);
    mostrarMedicos(C1);

    //...

    return 0;
}
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

Hospital

Este TAD representa los pacientes (código del paciente) ingresados en el mismo. La gravedad de los pacientes se indica mediante un dígito decimal (en orden decreciente de gravedad). Si un paciente tiene una gravedad entre 1 y 5, estará ingresado en la U.C.I. (Unidad de cuidados intensivos), si su gravedad se encuentra entre 6 y 9, estará en planta. Sobre decir que la gravedad 0 implica que el paciente ingresa en la morgue. Obviamente, la capacidad de la U.C.I. y de la planta es limitada, por lo tanto, si un paciente tiene que ingresar en la U.C.I. y ésta se encuentra completamente ocupada, el paciente menos grave de la misma pasará a planta, y si sucede lo mismo con un ingreso en planta, se dará de alta al paciente con mejor estado de salud. En situación anómala (tener pacientes con gravedad U.C.I. en planta), será resuelta lo antes posible, volviendo dichos pacientes a U.C.I. a la primera oportunidad.

Las operaciones del TAD serán las siguientes:

- Crear Hospital
- Ingreso
- Alta
- Muerte
- Pacientes en U.C.I
- Pacientes en Planta
- Pacientes de una gravedad determinada
- Destruir Hospital

a) Realice la especificación del TAD

b) Diseñe una estructura de datos adecuada para representar el TAD e implemente las operaciones anteriores.

NOTA: Es absolutamente necesario definir todos los tipos de datos implicados en la resolución del ejercicio, así como los prototipos de las operaciones utilizadas de los TADs conocidos.

```
#ifndef HOSPITAL_HPP
#define HOSPITAL_HPP
#include "../TADS Lista/listavec.hpp"
#include <iostream>
using namespace std;

// Definimos el tipo de dato "paciente"
struct Paciente
{
    int codPaciente_;
    int gravedad_;
    static int numPacientes_;
    Paciente(int gravedad): codPaciente_(numPacientes_++), gravedad_(gravedad){}
};

int Paciente::numPacientes_ = 0;
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
class Hospital{
public:
    Hospital(int maxUCI,int maxPlanta);
    void Ingreso(Paciente x);          //insertar
    void Alta(Paciente x);             //eliminar
    void Muerte(Paciente x);           //eliminar
    //Consultar
    const int PacientesUCI() const;
    const int PacientesPlanta() const;
    const int PacientesGravedad(const int grave) const;
    ~Hospital();

private:
    const int MaxUCI, MaxPlanta;
    Lista<Paciente> UCI, Planta;
    Lista<Paciente>::posicion buscarUCI(Paciente x);          //Metodo privado
    Lista<Paciente>::posicion buscarPlanta(Paciente x);       //Metodo privado
    Lista<Paciente>::posicion MenosGraveUCI();                //Metodo privado
    Lista<Paciente>::posicion MenosGravePlanta();              //Metodo privado
};

// <-- Metodo privado --> //

Lista<Paciente>::posicion Hospital::buscarUCI(Paciente x)
{
    //Buscar paciente menos grave en UCI.
    Lista<Paciente>::posicion pos = UCI.primer();
    bool encontrado = false;
    //Si su gravedad sobrepasa 5, no se busca en la UCI
    if(x.gravedad_ >0 && x.gravedad_ <= 5)
    {
        while((pos != UCI.fin()) && (!encontrado))
        {
            if(x.codPaciente_ == UCI.elemento(pos).codPaciente_)
                {encontrado = true;}
            else    {pos = Planta.siguiete(pos);}
        }
    }

    if(!encontrado)
    {
        pos = Planta.primer();
        while((pos != Planta.fin()) && (!encontrado))
        {
            if(x.codPaciente_ == Planta.elemento(pos).codPaciente_)
                {encontrado = true;}
            else    {pos = Planta.siguiete(pos);}
        }
    }
    return pos;
}

Lista<Paciente>::posicion Hospital::buscarPlanta(Paciente x)
{
    //Buscar paciente menos grave en UCI.
    Lista<Paciente>::posicion pos = UCI.primer();
    bool encontrado = false;

    pos = Planta.primer();
    while((pos != Planta.fin()) && (!encontrado))
    {
        if(x.codPaciente_ == Planta.elemento(pos).codPaciente_)
            {encontrado = true;}
        else    {pos = Planta.siguiete(pos);}
    }

    return pos;
}
```

```

Lista<Paciente>::posicion Hospital::MenosGraveUCI()
{
    int paciente_menos_grave = -100;
    Lista<Paciente>::posicion p = UCI.primer(); //Iterador
    Lista<Paciente>::posicion pmgUCI = UCI.primer(); //pmgUCI guarda la posicion del paciente de menor gravedad.

    while(p != UCI.fin())
    {
        if(UCI.elemento(p).gravedad_ > paciente_menos_grave)
        {
            paciente_menos_grave = UCI.elemento(p).gravedad_; //Guardamos menor gravedad
            pmgUCI = p; //Guardamos posición del paciente
        }
        p = UCI.siguiente(p);
    }
    return pmgUCI;
}

Lista<Paciente>::posicion Hospital::MenosGravePlanta()
{
    int paciente_menos_grave = -100;
    Lista<Paciente>::posicion p = Planta.primer(); //Iterador
    Lista<Paciente>::posicion pmgPlanta = Planta.primer();
    //pmgPlanta guarda la posicion del paciente de menor gravedad.

    while(p != Planta.fin())
    {
        if(Planta.elemento(p).gravedad_ > paciente_menos_grave)
        {
            paciente_menos_grave = Planta.elemento(p).gravedad_; //Guardamos menor gravedad
            pmgPlanta = p; //Guardamos poscion del paciente
        }
        p = Planta.siguiente(p);
    }
    return pmgPlanta;
}

// <-- CONSTRUCTOR --> //
Hospital::Hospital(int maxUCI,int maxPlanta): UCI(maxUCI), Planta(maxPlanta), MaxUCI(maxUCI), MaxPlanta(maxPlanta){}

// <-- INGRESAR --> //
void Hospital::Ingreso(Paciente x)
{
    switch(x.gravedad_)
    {
        case 0:
            std::cout << "Lamentamos la perdida del paciente" << x.codPaciente_ << "." << std::endl;
            break;
        case 1:case 2:case 3:case 4:case 5:
        {
            //Si no está llena la UCI
            if(!UCI.llena())
            {
                UCI.insertar(x,UCI.fin());
                std::cout << "Paciente " << x.codPaciente_ << " insertado en U.C.I." << std::endl;
            }
            //Si está llena la UCI
            else
            {
                //Buscar paciente menos grave en UCI.
                Lista<Paciente>::posicion pmgUCI = MenosGraveUCI();

                //Si nuestro paciente es el de menor gravedad entre los pacientes de la UCI
                if(x.gravedad_ >= UCI.elemento(pmgUCI).gravedad_)
                {
                    //Si no está llena la planta
                    if(!Planta.llena())
                    {
                        Planta.insertar(x,Planta.fin()); //Insertamos nuevo paciente
                        std::cout << "Paciente " << x.codPaciente_ << " insertado en Planta por falta
                            de espacio en UCI" << std::endl;
                    }
                }
            }
            else
            {
                //Buscar paciente menos grave en Planta.
                Lista<Paciente>::posicion pmgPlanta = MenosGravePlanta();
            }
        }
    }
}

```

```

        std::cout << "Paciente " << Planta.elemento(pmgPlanta).codPaciente_ << " dado
        de alta por falta de habitaciones." << std::endl;
        Planta.eliminar(pmgPlanta);
        Planta.insertar(x, Planta.fin());
        std::cout << "Paciente " << x.codPaciente_ << " insertado en Planta por falta
        de espacio en UCI" << std::endl;
    }
}
//Si nuestro paciente necesita entrar en la UCI
else
{
    //Si no está llena la planta
    if(!Planta.llena())
    {
        Planta.insertar(UCI.elemento(pmgUCI),Planta.fin());
        //Insertamos pmgUCI en planta
        std::cout << "Paciente " << UCI.elemento(pmgUCI).codPaciente_ << " insertado en
        Planta por falta de espacio en UCI" << std::endl;
        UCI.eliminar(pmgUCI); //Eliminamos pmgUCI en UCI
        UCI.insertar(x,UCI.fin()); //Insertamos nuevo paciente
        std::cout << "Paciente " << x.codPaciente_ << " insertado en U.C.I." <<
        std::endl;
    }
    //Si está llena la planta
    else
    {
        //Buscar paciente menos grave en Planta.
        Lista<Paciente>::posicion pmgPlanta = MenosGravePlanta();
        std::cout << "Paciente " << Planta.elemento(pmgPlanta).codPaciente_ << " dado
        de alta por falta de habitaciones." << std::endl;
        Planta.eliminar(pmgPlanta);
        Planta.insertar(pmgUCI, Planta.fin());
        std::cout << "Paciente " << UCI.elemento(pmgUCI).codPaciente_ << " insertado en
        Planta por falta de espacio en UCI" << std::endl;
        UCI.insertar(x,UCI.fin()); //Insertamos nuevo paciente
        std::cout << "Paciente " << x.codPaciente_ << " insertado en U.C.I." <<
        std::endl;
    }
}
}
}
break;
case 6:case 7:case 8:case 9:
{
    if(!Planta.llena())
    {
        Planta.insertar(x,Planta.fin()); //Insertamos nuevo paciente
        std::cout << "Paciente " << x.codPaciente_ << " insertado en Planta" << std::endl;
    }
    else
    {
        //Buscar paciente menos grave en Planta.
        Lista<Paciente>::posicion pmgPlanta = MenosGravePlanta();

        //Si nuestro paciente es el de menor gravedad entre los pacientes de la Planta
        if(x.gravedad_ >= Planta.elemento(pmgPlanta).gravedad_)
        {
            std::cout << "No podemos admitirle en el hospital por falta de habitaciones" << std::endl;
        }
        else
        {
            std::cout << "Paciente " << Planta.elemento(pmgPlanta).codPaciente_ << " dado de alta
            por falta de habitaciones." << std::endl;
            Planta.eliminar(pmgPlanta);
            Planta.insertar(x, Planta.fin());
            std::cout << "Paciente " << x.codPaciente_ << " insertado en Planta" << std::endl;
        }
    }
}
}
break;
default:
    std::cout << "No necesita ayuda hopsitalaria."<< std::endl;
    break;
}
}
}

```

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
// <-- ALTA --> //
void Hospital::Alta(Paciente x)
{
    //Buscar paciente
    Lista<Paciente>::posicion p = buscarUCI(x);

    //Si el paciente se encuentra en la UCI
    if(p != UCI.fin())
    {
        //Dar de alta al paciente
        std::cout << "Paciente" << x.codPaciente_ << "dado de alta." << std::endl;
        UCI.eliminar(p);

        //Buscar en planta paciente de mayor gravedad (más cercano a 1)
        p = Planta.primer(); //Iterador
        int paciente_mas_grave_Planta = 6; //Gravedad del paciente que se traspasará.
        Lista<Paciente>::posicion pmgPlanta = Planta.primer(); //Paciente que se traspasa a UCI

        while(p != Planta.fin())
        {
            //Si el paciente está más grave que el guardado.
            if(Planta.elemento(p).gravedad_ < paciente_mas_grave_Planta)
            {
                //Se actualiza la mayor gravedad
                paciente_mas_grave_Planta = Planta.elemento(p).gravedad_;
                //Se guarda en pmgPlanta la posición del paciente.
                pmgPlanta = p;
            }
            p = Planta.siguiete(p); //Iterar
        }
        //Si tiene la gravedad necesaria para ser insertado en la UCI
        if(Planta.elemento(pmgPlanta).gravedad_ > 0 && Planta.elemento(pmgPlanta).gravedad_ <= 5)
        {
            UCI.insertar(pmgPlanta, UCI.fin()); //Se inserta en uci a pmgPlanta
            Planta.eliminar(pmgPlanta); //Y se elimina de la planta
        }
    }
    //Si el paciente se encuentra en Planta
    else
    {
        p = buscarPlanta(x);
        if(p != Planta.fin())
        {
            std::cout << "Paciente" << x.codPaciente_ << "dado de alta." << std::endl;
            Planta.eliminar(p);
        }
    }
}

// <-- MUERTE --> //
void Hospital::Muerte(Paciente x)
{
    x.gravedad_ = 0;
    Lista<Paciente>::posicion p = buscarUCI(x);
    if(p != UCI.fin()) { UCI.eliminar(p); }
    else
    {
        p = buscarPlanta(x);
        if(p != Planta.fin()) {Planta.eliminar(p);}
    }
    std::cout << "Lamentamos la perdida del paciente" << x.codPaciente_ << "." << std::endl;
}

// <-- CONTAR PACIENTES EN UCI --> //
const int Hospital::PacientesUCI() const
{
    int cont = 0;
    Lista<Paciente>::posicion p = UCI.primer();

    while(p != UCI.fin())
    {
        cont++;
    }
    return cont;
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
// <-- CONTAR PACIENTES EN PLANTA --> //
const int Hospital::PacientesPlanta() const
{
    int cont = 0;
    Lista<Paciente>::posicion p = Planta.primer();

    while(p != Planta.fin())
    {
        cont++;
    }
    return cont;
}

// <-- CONTAR PACIENTES DE CIERTA GRAVEDAD --> //
const int Hospital::PacientesGravedad(const int grave) const
{
    int cont = 0;
    Lista<Paciente>::posicion p;
    //En planta puede haber pacientes de UCI, pero no viceversa.
    if(grave <= 5) //Si "grave" es menor o igual a 5
    {
        //Buscamos en UCI
        p = UCI.primer();
        while(p != UCI.fin())
        {
            if (UCI.elemento(p).gravedad_ == grave) {cont++;}
            p = UCI.siguiente(p);
        }

        //Buscamos en Planta
        p = Planta.primer();
        while(p != Planta.fin())
        {
            if (Planta.elemento(p).gravedad_ == grave) {cont++;}
            p = Planta.siguiente(p);
        }

        return cont;
    }
}

// <-- DESTRUCTOR --> //
Hospital::~Hospital()
{
    UCI.~Lista();
    Planta.~Lista();
    delete this;
}

#endif //HOSPITAL_HPP
```

Tres en Raya

TAD TRES EN RAYA

Representar TAD tablero tres en raya

Funciones:

- Crear tablero vacío
- Colocar ficha de un jugador en la casilla determinada
- Libres un conjunto de las casillas libres
- Libres la ficha de una casilla
- Comprobar si un jugador determinado ha ganado la partida
- Comprobar si ha terminado en empate
- Destruir tablero

```

/*****
DEFINICION:
    Un Tablero es una secuencia ordenada de casillas.

OPERACIONES:
    Tablero()
        Postcondiciones: Crea un tablero vacío

    void colocar_ficha(Ficha ficha, const casilla& casilla)
        Precondiciones: La casilla está en un lugar válido,
                        la ficha es X u O y la casilla está vacía.
                        La partida no debe haber acabado.
        Postcondiciones: Coloca la ficha en el lugar indicado.

    Lista<casilla> casillas_libres() const
        Postcondiciones: Devuelve una Lista con las casillas libres.

    Ficha obtener_ficha_casilla(const casilla& casilla) const
        Precondiciones: La casilla está en un lugar válido.
        Postcondiciones: Devuelve el Ficha de la casilla especificada.

    bool comprobar_ganador(Ficha jugador) const
        Precondiciones: Ficha no es Vacío.
        Postcondiciones: Devuelve verdadero si ha ganado el jugador
                        especificado y falso si no.

    bool comprobar_empate() const
        Precondiciones: La partida debe haber terminado.
                        Es decir, ninguna casilla puede estar vacía.
        Postcondiciones: Devuelve verdadero si hay un empate y falso si no.
*****/

#ifndef TRES_EN_RAYA
#define TRES_EN_RAYA
#include "../TADS Lista/listaenla.hpp"
#include <iostream>
#include <cassert>
using namespace std;

struct casilla
{
    size_t fila, columna;
    casilla(size_t f = 0, size_t c = 0) : fila {f}, columna {c} {}
};

enum class Ficha{Vacío, X, O};

```

```

class Tablero
{
    public:
        Tablero(); //Constructor.
        void colocar_ficha(Ficha ficha, const casilla& casilla); //Colocar ficha.
        Lista<casilla> casillas_libres() const; //Listar casillas libres.
        Fichas obtener_ficha_casilla(const casilla& casilla) const; //Ver ficha en casilla especifica.
        bool comprobar_ganador(Ficha jugador) const; //Comprobar que un jugador ha ganado.
        bool comprobar_empate() const; //Comprobar empate.
        ~Tablero() = default; //Destructor

    private:
        Fichas tablero[3][3]; //Tablero de Fichas. Matriz tipo Fichas
};

// <-- CONSTRUCTOR --> //
Tablero::Tablero()
{
    for (size_t i = 0; i < 3; i++)
        for (size_t j = 0; j < 3; j++)
            {tablero[i][j] = Fichas::Vacio;}
}

// <-- COLOCAR FICHA --> //
void Tablero::colocar_ficha(Ficha ficha, const casilla& casilla)
{
    //Casilla Valida.
    assert(casilla.fila < 3 && casilla.columna < 3);
    //Ficha a meter no vacía y Casilla Vacía.
    assert(ficha != Fichas::Vacio && tablero[casilla.fila][casilla.columna] == Fichas::Vacio);
    //La partida no ha acabado.
    assert(comprobar_ganador(Ficha::X) == false && comprobar_ganador(Ficha::O) == false);
    //Colocar ficha en tablero.
    tablero[casilla.fila][casilla.columna] = ficha;
}

// <-- CASILLAS LIBRES --> //
Lista<casilla> Tablero::casillas_libres() const
{
    Lista<casilla> Libres;

    for (size_t i = 0; i < 3; i++)
        for (size_t j = 0; j < 3; j++)
            if (tablero[i][j] == Fichas::Vacio) //Si la casilla está vacía
                Libres.insertar(casilla(i, j), Libres.fin()); //Inserta en la lista

    return Libres;
}

// <-- OBTENER FICHA --> //
Fichas Tablero::obtener_ficha_casilla(const casilla& casilla) const
{
    //Casilla Valida.
    assert(casilla.fila < 3 && casilla.columna < 3);
    //Devuelve ficha en esa posición.
    return tablero[casilla.fila][casilla.columna];
}

bool Tablero::comprobar_empate() const
{
    size_t contador = 0;
    for (size_t i = 0; i < 3; i++)
        for (size_t j = 0; j < 3; j++)
            if (tablero[i][j] != Fichas::Vacio)
                contador++;

    // Ninguna casilla puede estar vacía.
    assert(contador == 9);
    //Si ambos son falsos, han empatado.
    return !comprobar_ganador(Ficha::X) && !comprobar_ganador(Ficha::O);
}

```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...
ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
bool Tablero::comprobar_ganador(Ficha jugador) const
{
    assert(jugador != Ficha::Vacio);
    bool ganador = false;
    size_t contador = 0;

    // columnas
    for (size_t i = 0; i < 3; i++)          //Columna
    {
        for (size_t j = 0; j < 3; j++)      //Fila
        {
            if (tablero[i][j] == jugador)  contador++;
        }

        if (contador == 3)                  {ganador = true;}          //Tres en raya (columna)
        else                                {contador = 0;}
    }

    if(!ganador)
    {
        // filas
        for (size_t i = 0; i < 3; i++)      //Fila
        {
            for (size_t j = 0; j < 3; j++)  //Columna
            {
                if (tablero[j][i] == jugador) contador++;
            }

            if (contador == 3)              {ganador = true;}          //Tres en raya (fila)
            else                            {contador = 0;}
        }

        if(!ganador)
        {
            // Diagonal AI -> BD (Arriba IZQ Abajo DER)
            for (size_t i = 0; i < 3; i++)
            {
                if (tablero[i][i] == jugador)  contador++;
            }

            if (contador == 3)              {ganador = true;}
            else                            {contador = 0;}

            if(!ganador)
            {
                // Diagonal AD -> BI (Arriba DER Abajo IZQ)
                for (size_t i = 0, j = 2; i < 3; i++, j--)
                {
                    if (tablero[i][j] == jugador)  contador++;
                }
            }

            if (contador == 3)              {ganador = true;}
        }
    }

    return ganador;
}

std::ostream& operator<<(std::ostream& os, const Lista<casilla>& l)
{
    auto pos = l.primera();
    while (pos != l.fin())
    {
        os << "\t{ " << l.elemento(pos).fila << ", " << l.elemento(pos).columna << " }\n";
        pos = l.siguiente(pos);
    }
    return os;
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
std::ostream& operator<<(std::ostream& os, const Tablero& T)
{
    for (size_t i = 0; i < 3; i++){
        for (size_t j = 0; j < 3; j++){
            switch(T.obtener_ficha_casilla({ i, j })){
                case Ficha::Vacio:
                    os << "|  |";
                    break;
                case Ficha::X:
                    os << "| X |";
                    break;
                case Ficha::O:
                    os << "| O |";
                    break;
            }
        }
        os << std::endl;
    }
    return os;
}

#endif // TRES_EN_RAYA

////////////////////////////////////

#include "tres_en_raya.hpp"

int main()
{
    Tablero t;

    for (size_t i = 0; i < 3; i++)
        for (size_t j = 0; j < 3; j++)
            assert(t.obtener_ficha_casilla({ i, j }) == Ficha::Vacio);

    assert(!t.comprobar_ganador(Ficha::X));
    assert(!t.comprobar_ganador(Ficha::O));

    std::cout << "Tablero nuevo: " << endl << t << "-----" << endl;
    std::cout << "Fichas vacias:\n" << t.casillas_libres() << "-----" << endl;

    // Hacemos que gane X, diagonal 1
    t.colocar_ficha(Ficha::X, { 0, 0 });
    t.colocar_ficha(Ficha::X, { 1, 1 });
    t.colocar_ficha(Ficha::X, { 2, 2 });

    assert(t.comprobar_ganador(Ficha::X));
    assert(!t.comprobar_ganador(Ficha::O));

    std::cout << "Gana X: " << endl << t << "-----" << endl;
    std::cout << endl;
    std::cout << "Fichas vacias al ganar X:\n" << t.casillas_libres();
    std::cout << "-----" << endl;

    // Reiniciamos el tablero.
    t = Tablero {};
    std::cout << "Tablero nuevo: " << endl << t << "-----" << endl;
    std::cout << "Fichas vacias:\n" << t.casillas_libres() << "-----" << endl;

    // Empatamos
    t.colocar_ficha(Ficha::X, { 0, 2 });
    t.colocar_ficha(Ficha::X, { 1, 0 });
    t.colocar_ficha(Ficha::X, { 2, 1 });
    t.colocar_ficha(Ficha::X, { 2, 2 });

    t.colocar_ficha(Ficha::O, { 0, 0 });
    t.colocar_ficha(Ficha::O, { 0, 1 });
    t.colocar_ficha(Ficha::O, { 1, 1 });
    t.colocar_ficha(Ficha::O, { 1, 2 });
    t.colocar_ficha(Ficha::O, { 2, 0 });

    assert(!t.comprobar_ganador(Ficha::X));
    assert(!t.comprobar_ganador(Ficha::O));
    assert(t.comprobar_empate());

    std::cout << "Tablero empate: " << endl << t << "-----" << endl;
    std::cout << "Fichas vacias:\n" << t.casillas_libres() << "-----" << endl;
}
```

```
// Reiniciamos el tablero.
t = Tablero {};

// Hacemos que gane O, filas
t.colocar_ficha(Ficha::O, { 0, 0 });
t.colocar_ficha(Ficha::O, { 1, 0 });
t.colocar_ficha(Ficha::O, { 2, 0 });

assert(!t.comprobar_ganador(Ficha::X));
assert(t.comprobar_ganador(Ficha::O));

// Reiniciamos el tablero.
t = Tablero {};
std::cout << "Tablero nuevo: " << endl << t << "-----" << endl;
std::cout << "Fichas vacias:\n" << t.casillas_libres() << "-----" << endl;

// Hacemos que gane X, columnas
t.colocar_ficha(Ficha::X, { 0, 0 });
t.colocar_ficha(Ficha::X, { 0, 1 });
t.colocar_ficha(Ficha::X, { 0, 2 });

assert(t.comprobar_ganador(Ficha::X));
assert(!t.comprobar_ganador(Ficha::O));

std::cout << "Gana X: " << endl << t << "-----" << endl;
std::cout << endl;
std::cout << "Fichas vacias al ganar X:\n" << t.casillas_libres();
std::cout << "-----" << endl;

// Reiniciamos el tablero.
t = Tablero {};
std::cout << "Tablero nuevo: " << endl << t << "-----" << endl;
std::cout << "Fichas vacias:\n" << t.casillas_libres() << "-----" << endl;

// Hacemos que gane O, diagonal 2
t.colocar_ficha(Ficha::O, { 0, 2 });
t.colocar_ficha(Ficha::O, { 1, 1 });
t.colocar_ficha(Ficha::O, { 2, 0 });

assert(!t.comprobar_ganador(Ficha::X));
assert(t.comprobar_ganador(Ficha::O));

std::cout << "Gana O: " << endl << t << "-----" << endl;
std::cout << endl;
std::cout << "Fichas vacias al ganar O:\n" << t.casillas_libres();
std::cout << "-----" << endl;

return 0;
}
```

Hipermercado/Línea de Cajas

Un hipermercado dispone para el pago de las compras de 50 cajas numeradas del 1 al 50. El número de cajas que permanecen abiertas a lo largo del día es variable, de tal forma que en cada momento hay entre 10 y 50 cajas en funcionamiento, dependiendo de la afluencia de clientes (en las horas y fechas puntas hay más cajas abiertas que en momentos de menor clientela), aunque la media de cajas abiertas es de 48. El horario de venta se cubre con dos turnos de trabajo al día y en cada turno se supone que hay suficientes cajeros para atender la demanda máxima que se pueda producir y además algunos suplentes para sustituir a los que necesiten descansar. Para cada caja hay que almacenar el número de caja, el identificador del cajero(3 dígitos) y la facturación realizada desde el comienzo del turno de trabajo.

a) Escribe la especificación del TAD LíneaCajas con las siguientes operaciones:

- Crear una línea de cajas cerradas
- Abrir una caja determinada
- Cerrar una caja determinada y devolver la facturación realizada en ella desde la última vez que se abrió
- Cobrar a un cliente el importe de su compra en una caja.
- Sustituir al cajero de una caja determinada. La caja continúa funcionando
- Cambiar el turno de trabajo y devolver la recaudación total que hay en las cajas en el momento del cambio. En esta operación puede que el total de cajas, así como las cajas concretas de uno y otro turno no coincidan
- Cerrar todas las cajas y devolver la recaudación total que hay en ellas en el momento del cierre

b) Realiza el TAD teniendo en cuenta lo siguiente:

- El tiempo de ejecución de cerrar una caja, cobrar una compra y sustituir a un cajero debe de ser constante

Justifica razonadamente la estructura de datos elegida en términos de eficiencia en tiempo y espacio.

/*****

Usamos un vector para almacenar los objetos de las Cajas, ya que el enunciado exige que el tiempo de ejecución de cerrar una caja, cobrar una compra y sustituir a un cajero debe ser constante. Esto solo es posible haciendo uso de un vector, ya que el TAD Lista no provee acceso directo a sus elementos, sino que habría que recorrer cada uno de ellos hasta llegar al que interesa. Esto, evidentemente, no se puede realizar en tiempos constantes en el peor caso. Además, la media de cajas abiertas (en uso) son de 48/50, dando así la información de poco gasto de memoria.

Por lo tanto, la única opción posible para realizar la implementación del TAD LíneaCajas es usando un vector con capacidad de hasta 50 cajas.

ESPECIFICACIÓN:

Una Caja es una estructura de datos que almacena el identificador de un cajero, la facturación desde su apertura y un booleano que indica si dicha Caja está abierta o no.

Una LíneaCajas es una secuencia de Cajas que permite añadir compras de clientes a Cajas, así como abrirlas, cerrarlas, cambiar turnos y obtener la recaudación.

OPERACIONES:

LíneaCajas()

Postcondiciones: Crea una línea de cajas cerradas vacía.

void abrir_caja(int numero_caja, size_t identificador_cajero)

Precondiciones: La Caja con el identificador especificado existe y no está abierta.

Postcondiciones: Abre la Caja con el identificador especificado.

double cerrar_caja(size_t numero_caja)

Precondiciones: La Caja con el identificador especificado existe y está abierta.

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...
ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
Postcondiciones: Cierra la Caja y devuelve su recaudación desde el momento en la que se abrió. Se vacía la caja.

void cobrar_cliente(size_t numero_caja, double importe)
    Precondiciones: La Caja con el identificador especificado existe y está abierta.
    Postcondiciones: Añade el importe a la recaudación de la Caja especificada.

void sustituir_cajero(size_t numero_caja, size_t identificador_cajero)
    Precondiciones: La Caja con el identificador especificado existe y está abierta.
    Postcondiciones: Sustituye el cajero por el especificado.

double cambiar_turno()
    Postcondiciones: Devuelve la suma de las recaudaciones de cada Caja desde el momento en el que se abrieron cada una. No cierra ninguna Caja, pero las reinicia a 0.

double cerrar_cajas()
    Postcondiciones: Cierra todas las Cajas y devuelve la suma de todas sus recaudaciones desde el momento en el que se abrieron. Se reinician a 0.
*****/

struct Caja
{
    //int idCaja;           //Al ser un vector de cajas, la posicion del vector es el número de caja.
    size_t idCajero;       //3 digitos
    double facturacion;
    bool abierta;          //True = abierta False = cerrada

    Caja(size_t cajero = 0, double caja_facturacion = 0.0, bool caja_abierta = false) :
        idCajero { cajero }, facturacion { caja_facturacion }, abierta { caja_abierta } { }
};

class LineaCajas
{
public:
    LineaCajas() = default;
    void abrirCaja(int numC, size_t id);
    double cerrarCaja(int numC);           //Devuelve facturacion
    void cobrar(double total, int numC);
    void sustituirCajero(int numC, size_t id);
    double cambiarTurno();                //Devuelve facturacion
    double cerrarCajas();                 //Devuelve facturacion
    ~LineaCajas() = default;
private:
    static const int max_cajas = 50;
    Caja cajas[max_cajas];
    //Para las operaciones que deben ser constantes, el acceso a la caja correspondiente debe ser constante
};

void LineaCajas::abrirCaja(int numC, size_t id)
{
    //Que la caja exista y no esté abierta.
    assert(numC < max_cajas && !cajas[numC].abierta);

    cajas[numC].abierta = true;
    cajas[numC].facturacion = 0.0;
    cajas[numC].idCajero = id;
}

double LineaCajas::cerrarCaja(int numC)
{
    //Que la caja exista y esté abierta.
    assert(numC < max_cajas && cajas[numC].abierta);

    cajas[numC].abierta = false;
    double Fact = cajas[numC].facturacion;
    cajas[numC].facturacion = 0.0;           //Al cerrar caja vuelve a 0.
    cajas[numC].idCajero = 0;

    return Fact;
    //return cajas[numC].facturacion;
}

void LineaCajas::cobrar(double total, int numC)
{
    //Que la caja exista y esté abierta.
    assert(numC < max_cajas && cajas[numC].abierta);

    cajas[numC].facturacion = cajas[numC].facturacion + total;
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
void LineaCajas::sustituirCajero(int numC, size_t id)
{
    //Que la caja exista y esté abierta.
    assert(numC < max_cajas && cajas[numC].abierta);

    cajas[numC].idCajero = id;
}

double LineaCajas::cambiarTurno()
{
    double Fact = 0.0;

    for (int numC = 0; numC < max_cajas; numC++)
    {
        if(cajas[numC].abierta)
        {
            Fact += cajas[numC].facturacion;
            cajas[numC].facturacion = 0.0;
        }
    }
    return Fact;
}

double LineaCajas::cerrarCajas()
{
    double Fact = 0.0;
    for (int numC = 0; numC < max_cajas; numC++)
    {
        if (cajas[numC].abierta)
        {
            Fact += cajas[numC].facturacion;
            cajas[numC].facturacion = 0.0; //Al cerrar caja vuelve a 0.
            cajas[numC].idCajero = 0;
            cajas[numC].abierta = false;
        }
    }
    return Fact;
}

#endif //HIPERMERCADO_HPP

int main()
{
    LineaCajas l;

    //void abrirCaja(int numC, size_t id);
    l.abrirCaja(1, 1);
    l.abrirCaja(2, 2);

    //void cobrar(double total, int numC);
    l.cobrar(1.5, 1);
    std::cout << "Cambio de turno: " << l.cambiarTurno() << std::endl;

    l.cobrar(3.5, 2);
    std::cout << "Cerrar caja 2: " << l.cerrarCaja(2) << std::endl;

    //void sustituirCajero(int numC, size_t id);
    l.sustituirCajero(1,3);
    l.abrirCaja(2, 4);

    l.cobrar(3.3, 1);
    l.cobrar(6, 1);
    l.cobrar(7.2, 1);

    l.cobrar(4.4, 2);
    l.cobrar(6.6, 2);

    l.abrirCaja(3, 2);

    //double cerrarCajas();
    std::cout << "Cerrar cajas: " << l.cerrarCajas() << std::endl;

    return 0;
}
```

Gestor de Impresión

En una oficina tienen una impresora que sirve a distintos usuarios, cada uno de los cuales puede enviar a la misma dos clases de trabajos, urgentes y no urgentes. Los trabajos de la misma clase de un usuario concreto serán procesados por orden de llegada y siempre los urgentes antes que los que no lo sean.

Asumiremos que los trabajos se identifican mediante un código único de tipo string.

La impresora atiende a los usuarios por turnos, de forma que al que le llega el turno imprime tan sólo uno de sus trabajos, a continuación, pasa el turno al siguiente usuario en espera y, si tiene trabajos pendientes, esperará a que le vuelva a llegar el turno después de que lo reciban todos los usuarios que ya estén esperando. Cuando un usuario envía su primer trabajo de impresión se le asigna el turno que sigue al del último en espera con trabajos pendientes. Un usuario pierde su turno al terminar de imprimir todos sus trabajos y se le asignará otro turno la próxima vez que solicite imprimir un nuevo trabajo.

a) Especifica las siguientes operaciones del TAD Gestor de Impresión:

- Crear un gestor de impresión para n usuarios.
- Añadir un trabajo de un usuario y de una urgencia determinada.
- Eliminar el trabajo a imprimir.
- Cancelar todos los trabajos de un usuario.

b) Implementa el TAD Gestor de Impresión según la especificación del apartado anterior, justificando razonadamente la estructura de datos empleada.

NOTA 1 Se prohíbe expresamente la utilización del TAD Lista para la resolución del problema.

NOTA 2 Es absolutamente necesario definir todos los tipos implicados en la resolución del problema, así como los prototipos de las operaciones utilizadas de los TAD conocidos.

```
#ifndef GESTOR_IMPRESION_H
#define GESTOR_IMPRESION_H
#include <iostream>
#include <cstdio>
#include <cstring>
#include <vector>
#include "colaenla.hpp"
using namespace std;

typedef string trabajo;
//enum Urgencia {NO_URGE, URGE};

struct Usuario
{
    Cola<trabajo> URGENTE;
    Cola<trabajo> NO_URGENTE;
};

class Gestor{
public:
    Gestor(int num_users);
    void verGestor();
    int verEmpleados();
    void insertar_trabajo(int user_id, const trabajo codigo_trabajo, bool Urgencia);
    void eliminar_trabajo();
    void cancelar_trabajo(int user_id);

private:
    int empleados;
    std::vector<Usuario> Trabajadores;
    Cola<int> Impresora;
};

Gestor::Gestor(int num_users): Trabajadores(num_users), empleados(num_users){}
```

```
//No funciona :(
void Gestor::verGestor()
{
    Cola<int> copiaImpresora(Impresora); // Copia la cola original

    std::cout << "La cola de turnos: ";
    while (!copiaImpresora.vacia())
    {
        std::cout << copiaImpresora.frente() << " ";
        copiaImpresora.pop();
    }
    std::cout << std::endl;
}

int Gestor::verEmpleados()
{
    return empleados;
}

void Gestor::insertar_trabajo(int user_id, const trabajo codigo_trabajo, bool Urgencia)
{
    assert(user_id < empleados);

    //Añadir trabajo al usuario.
    if(Urgencia){
        Trabajadores[user_id].URGENTE.push(codigo_trabajo);
    }
    else{
        Trabajadores[user_id].NO_URGENTE.push(codigo_trabajo);
    }

    //Comprobar turno en cola Impresora
    if(!Impresora.vacia())
    {
        int og,aux;
        aux = og = Impresora.frente();

        if(og != user_id)
        {
            Impresora.pop();
            Impresora.push(aux);
            aux = Impresora.frente();
            while(aux != user_id && og != aux)
            {
                Impresora.pop();
                Impresora.push(aux);
                aux = Impresora.frente();
            }
            if(aux == og) //Ha dado toda la vuelta
                Impresora.push(user_id);
            else //Volver a la normalidad la cola
            {
                while(aux != og)
                {
                    Impresora.pop();
                    Impresora.push(aux);
                    aux = Impresora.frente();
                }
            }
        }
        //Si el id está al frente no se hace nada
    }
    else
    {
        Impresora.push(user_id);
        //std::cout << "Turno de: " << Impresora.frente() << std::endl;
    }
}

}
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
void Gestor::eliminar_trabajo()
{
    assert(!Impresora.vacia());

    int usu = Impresora.frente();

    //Si el trabajador tiene un trabajo pendiente en Urgente
    if(!Trabajadores[usu].URGENTE.vacia())
    {
        Trabajadores[usu].URGENTE.pop();
    }
    //Si el trabajador tiene un trabajo pendiente en No Urgente
    else if(!Trabajadores[usu].NO_URGENTE.vacia())
    {
        Trabajadores[usu].NO_URGENTE.pop();
    }

    //Pasar de turno.
    if(!Impresora.vacia()) Impresora.pop();

    //Devolverle a la cola si tiene trabajos pendientes
    if(!Trabajadores[usu].URGENTE.vacia() || !Trabajadores[usu].NO_URGENTE.vacia())
    {
        Impresora.push(usu);
    }
    //if(!Impresora.vacia()) std::cout << "Turno siguiente es de: " << Impresora.frente() << std::endl;
}

void Gestor::cancelar_trabajo(int user_id)
{
    //Borrar Trabajos
    while(!Trabajadores[user_id].URGENTE.vacia())
        Trabajadores[user_id].URGENTE.pop();
    while(!Trabajadores[user_id].NO_URGENTE.vacia())
        Trabajadores[user_id].NO_URGENTE.pop();

    //Comprobar turno en cola Impresora
    if(!Impresora.vacia())
    {
        int og, aux;
        aux = og = Impresora.frente();

        if(og != user_id)
        {
            Impresora.pop();
            Impresora.push(aux);
            aux = Impresora.frente();

            while(aux != user_id && og != aux)
            {
                Impresora.pop();
                Impresora.push(aux);
                aux = Impresora.frente();
            }

            if(aux != og) //No ha dado toda la vuelta
            {
                Impresora.pop();//Eliminamos el id del usuario.
                aux = Impresora.frente();
                while(aux != og)
                {
                    Impresora.pop();
                    Impresora.push(aux);
                    aux = Impresora.frente();
                }
            }
        }
        //Si el id está al frente se elimina el frente
        else Impresora.pop();
    }
}

#endif //GESTOR_IMPRESION_H
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

//

```
#include "gestordeimpresion.hpp"
#include <iostream>
#include <string>
#include <cassert>
int main()
{
    Gestor g { 10 };

    std::cout << g.verEmpleados() << " empleados."<< std::endl;

    trabajo a = "12534334";
    trabajo b = "128834334";
    trabajo c = "111534334";
    trabajo d = "1211134334";
    trabajo e = "1213334334";

    g.insertar_trabajo(2, a, false);
    g.insertar_trabajo(2, b, true);
    g.insertar_trabajo(3, c, false);
    g.insertar_trabajo(4, d, false);
    g.insertar_trabajo(3, e, true);

    g.verGestor();

    g.eliminar_trabajo();
    g.eliminar_trabajo();
    g.cancelar_trabajo(2);

    g.verGestor();

    return 0;
}
```

Diccionario

Crear diccionario ingles-español, en que palabra inglesa puede tener varias traducciones.

- Crear diccionario vacío.
- Insertar traducción español de una palabra inglesa.
- Eliminar una traducción en español de una palabra en inglés.
- Consultar si es correcta la traducción en español de una palabra en inglés.
- Consultar TODAS las traducciones al español de una inglesa.
- Destruir diccionario.

NOTA: Se permite el uso de string.h

```
#ifndef DICCIONARIO_HPP
#define DICCIONARIO_HPP
#include <iostream>
#include <cstring>
#include "listaenla.hpp"
using namespace std;    //Para que funciona string//typedef string palabra;

struct traduccion
{
    string ingles;
    Lista<string> espanol;

    traduccion (const string ing = {}) : ingles (ing){}
};

class Diccionario
{
public:
    Diccionario() = default;
    void Insertar(const string eng, const string esp);
    void Eliminar(const string eng, const string esp);
    bool ConsultarUna(const string eng, const string esp) const noexcept;
    Lista<string> ConsultarTodas(const string eng) const noexcept;
    ~Diccionario() = default;
private:
    Lista<traduccion> Palabras;
    typedef typename Lista<traduccion>::posicion posicion;
    typedef typename Lista<string>::posicion posiesp;
};

void Diccionario::Insertar(const string eng, const string esp)
{
    posicion pos = Palabras.primer();
    bool insertado = false;

    //En orden del diccionario
    while (pos != Palabras.fin() && !insertado)
    {
        // Si nos encontramos la palabra ya en el diccionario
        if(Palabras.elemento(pos).ingles == eng)
        {
            insertado = true;
        }
        else if(Palabras.elemento(pos).ingles != eng)
        {
            //O encontramos su lugar en el diccionario
            if (Palabras.elemento(pos).ingles > eng)
            {
                insertado = true;
            }
        }
        else pos = Palabras.siguiente(pos);
    }
}
```

```
//Si no se encuentra la palabra en ingles y es la de "mayor tamaño"
if (pos == Palabras.fin() /*|| !insertado*/)
{
    Palabras.insertar(traduccion{eng}, Palabras.fin());
    pos = Palabras.anterior(Palabras.fin());
}

// Insertar la traduccion
posiesp pos2 = Palabras.elemento(pos).espanol.primer();
insertado = false;

while (pos2 != Palabras.elemento(pos).espanol.fin() && !insertado)
{
    if(Palabras.elemento(pos).espanol.elemento(pos2) != esp)
    {
        if (Palabras.elemento(pos).espanol.elemento(pos2) > esp)
        {
            Palabras.elemento(pos).espanol.insertar(esp, pos2);
            insertado = true;
        }
    }
    pos2 = Palabras.elemento(pos).espanol.siguiete(pos2);
}
//Si no se ha insertado, se añade al final
if(!insertado) Palabras.elemento(pos).espanol.insertar(esp, Palabras.elemento(pos).espanol.fin());
}

void Diccionario::Eliminar(const string eng, const string esp)
{
    posicion pos = Palabras.primer();
    bool encontrado = false;

    while (pos != Palabras.fin() && !encontrado)
    {
        // Si nos encontramos la palabra ya en el diccionario
        if(Palabras.elemento(pos).ingles == eng)
        {
            encontrado = true;
        }
        else pos = Palabras.siguiete(pos);
    }

    if(encontrado)
    {
        // Eliminar la traduccion
        posiesp pos2 = Palabras.elemento(pos).espanol.primer();
        bool eliminado = false;

        while (pos2 != Palabras.elemento(pos).espanol.fin() && !eliminado)
        {
            // Si nos encontramos una traducción igual, la eliminamos.
            if(Palabras.elemento(pos).espanol.elemento(pos2) == esp)
            {
                Palabras.elemento(pos).espanol.eliminar(pos2);
                eliminado = true;
            }
            else pos2 = Palabras.elemento(pos).espanol.siguiete(pos2);
        }
    }
    //Si no se encuentra no se hace nada.
}

bool Diccionario::ConsultarUna(const string eng, const string esp) const noexcept
{
    posicion pos = Palabras.primer();
    bool encontrado = false;

    while (pos != Palabras.fin() && !encontrado)
    {
        // Si nos encontramos la palabra ya en el diccionario
        if(Palabras.elemento(pos).ingles == eng)
        {
            encontrado = true;
        }
        else pos = Palabras.siguiete(pos);
    }
}
```

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
if(encontrado)
{
    // Encontrar la traducción
    posiesp pos2 = Palabras.elemento(pos).espanol.primer();
    encontrado = false;

    while (pos2 != Palabras.elemento(pos).espanol.fin() && !encontrado)
    {
        // Si nos encontramos una traducción igual, la eliminamos.
        if(Palabras.elemento(pos).espanol.elemento(pos2) == esp)
        {
            encontrado = true;
        }
        pos2 = Palabras.elemento(pos).espanol.siguiente(pos2);
    }

    return encontrado;
}
else return encontrado;
}

Lista<string> Diccionario::ConsultarTodas(const string eng) const noexcept
{
    posicion pos = Palabras.primer();
    bool encontrado = false;

    while (pos != Palabras.fin() && !encontrado)
    {
        // Si nos encontramos la palabra ya en el diccionario
        if(Palabras.elemento(pos).ingles == eng)
        {
            encontrado = true;
        }
        else pos = Palabras.siguiente(pos);
    }

    if(encontrado)
    {
        Lista<string> copia(Palabras.elemento(pos).espanol);
        return copia;
    }
    else
    {
        std::cout << "No se ha encontrado la palabra" << eng << "en el diccionario" << std::endl;
        Lista<string> nada;
        return nada;
    }
}

#endif //DICCIONARIO_HPP
////////////////////////////////////

int main()
{
    Diccionario d;

    d.Insertar("hello", "finalmente");
    d.Insertar("hello", "buenas");
    d.Insertar("hello", "saludos");
    d.Insertar("hello", "hola");
    d.Insertar("hello", "holita");

    d.Eliminar("hello", "buenas");

    if(!d.ConsultarUna("hello", "buenas"))
        std::cout << "\"Buenas\" no se encuentra en el diccionario" << std::endl;

    if(d.ConsultarUna("hello", "saludos"))
        std::cout << "\"Saludos\" si se encuentra en el diccionario" << std::endl;

    const auto& traducciones = d.ConsultarTodas("hello");

    std::cout << "Traducciones para la palabra \"hello\": " << std::endl;
    for (auto pos = traducciones.primer(); pos != traducciones.fin(); pos = traducciones.siguiente(pos))
        std::cout << '\t' << traducciones.elemento(pos) << std::endl;

    return 0;
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Token Bus

El estándar IEEE 802.4 define las redes TokenBus. TokenBus combina una estructura de bus de las redes Ethernet y el sistema de testigo Token Ring. El modo de transmisión es el de banda ancha sobre cable coaxial. Los computadores están conectados a un bus, pero el trabajo se realiza en un anillo lógico. Cada máquina está identificada con un nombre (de máximo 8 caracteres) dentro de la red.

El testigo o token es una rama de información que otorga la posibilidad de enviar datos a la estación en la que se encuentre. El testigo se va desplazando en orden por la red, si una máquina recibe el testigo, puede transmitir información o bien pasarlo a la máquina siguiente.

El orden viene determinado por el orden de inserción de las máquinas en el bus. Por motivo de medio de transmisión el máximo de computadores es de 25.

El objetivo es diseñar un TAD con las siguientes operaciones:

- Inicializar la red, máximo de 25 computadores por red, añade el primer computador e inicializa el token
- Añadir computador
- Eliminar computador, sale el primero que entró en la red
- Pasar el token al siguiente
- Token: indica que computador tiene el token
- Enviar una trama concreta del computador A al computador B. Suponemos que trama es un tipo ya definido

```
#ifndef TOKEN_BUS_HPP
#define TOKEN_BUS_HPP
#include <iostream>
#include <cassert> // assert
#include <cstddef> // size_t
#include <cstring> // strncpy
#include "listacircular.hpp"
using namespace std; //Para que funciona string

typedef int trama;

//Cada máquina está identificada con un nombre (de máximo 8 caracteres)
struct Computador
{
    char nombre[8 + 1 /* null character */];
    bool tiene_token;
    trama tr;
    Computador(const char* nombre, const trama& t);
};

Computador::Computador(const char* nombre_computador, const trama& t): tiene_token { false }, tr { t }
{
    strncpy(nombre, nombre_computador, sizeof(nombre));
}
```

```
class TokenBus
{
public:
    TokenBus(const Computador& computador);
    void Insertar(const Computador& computador);
    void Eliminar();
    void PasarToken();
    const Computador& TieneToken();
    void EnviarTrama(const char* compA, const char* compB);
private:
    const size_t max_computadores = 25;
    size_t num_computadores;
    ListaCir<Computador> Computadores;

    typedef typename ListaCir<Computador>::posicion posicion;
    posicion primer_computador;
    posicion BuscarComputador(const char* nombre) const noexcept;
};

// <-- PRIVADO --> //
TokenBus::posicion TokenBus::BuscarComputador(const char* nombre) const noexcept
{
    posicion pos = Computadores.inipos();
    bool encontrado = false;
    do
    {
        //Si son iguales
        if (strcmp(Computadores.elemento(pos).nombre, nombre) == 0)
        {
            encontrado = true;
        }
        else
            pos = Computadores.siguiente(pos);
    } while (pos != Computadores.inipos() && !encontrado);

    if (encontrado) return pos;
    else
        return Computadores.POS_NULA;
}

// <-- CONSTRUCTOR --> //
TokenBus::TokenBus(const Computador& computador)
{
    num_computadores = 1;
    Computadores.insertar(computador, Computadores.inipos());
    primer_computador = Computadores.inipos();

    // Inicializamos el token.
    Computadores.elemento(primer_computador).tiene_token = true;
}

// <-- INSERTAR --> //
void TokenBus::Insertar(const Computador& computador)
{
    assert(++num_computadores <= max_computadores);
    // No podemos insertar un computador con token a no ser que sea el único
    assert(num_computadores == 1 || computador.tiene_token == false);
    // Nos aseguramos de que no hay ningún ordenador con el mismo identificador ya.
    assert(!BuscarComputador(computador.nombre));
    // Insertamos el computador al final. Anterior al inipos, es el final.

    Computadores.insertar(computador, Computadores.anterior(primer_computador));
}

// <-- ELIMINAR --> //
void TokenBus::Eliminar()
{
    assert(num_computadores != 0);
    // Si vamos a eliminar el computador con token, lo pasamos al siguiente. Si es el único computador que hay,
    // no quedará otra que hacer que dejemos de tener tokens.

    if (Computadores.elemento(primer_computador).tiene_token)
        PasarToken();
    posicion CompAeliminar = primer_computador;
    primer_computador = Computadores.siguiente(primer_computador);
    Computadores.eliminar(CompAeliminar);
    num_computadores--;
}
```

```
// <-- PASAR TOKEN --> //

void TokenBus::PasarToken()
{
    assert(num_computadores != 0);
    posicion pos = primer_computador;
    bool pasado = false;

    do
    {
        if (Computadores.elemento(pos).tiene_token)
        {
            Computadores.elemento(pos).tiene_token = false;
            Computadores.elemento(Computadores.siguiente(pos)).tiene_token = true;
            pasado = true;
            // Cuando hayamos encontrado el ordenador con el token, lo pasamos al siguiente y acabamos.
        }
        else pos = Computadores.siguiente(pos);
    } while (pos != primer_computador && !pasado);}

// <-- Ordenador con TOKEN --> //

const Computador& TokenBus::TieneToken()
{
    assert(num_computadores != 0);

    posicion pos = primer_computador;
    bool encontrado = false;

    do
    {
        if (Computadores.elemento(pos).tiene_token)
            encontrado = true;
        else pos = Computadores.siguiente(pos);
    }while (pos != primer_computador && !encontrado);

    return Computadores.elemento(pos);
}

// <-- ENVIAR TRAMA --> //
void TokenBus::EnviarTrama(const char* compA, const char* compB)
{
    posicion pos_a = BuscarComputador(compA), pos_b = BuscarComputador(compB);
    // Comprobamos que efectivamente existen ambos computadores.
    assert(pos_a != Computadores.POS_NULA && pos_b != Computadores.POS_NULA);
    // Evitamos auto-enviarse tramas.
    assert(pos_a != pos_b);
    // Nos aseguramos de que tiene el token el computador que va a enviar la trama.
    assert(Computadores.elemento(pos_a).tiene_token);

    Computadores.elemento(pos_b).tr = Computadores.elemento(pos_a).tr;
    //PasarToken();
}

#endif //TOKEN_BUS_HPP

////////////////////////////////////
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
/* PRUEBAS */

#include <iostream>
#include "tokenbus.hpp"

int main()
{
    TokenBus t { Computador { "c1", 1 } };

    std::cout << "Token creado" << std::endl;

    t.Insertar(Computador { "c2", { 2 } });
    t.Insertar(Computador { "c3", { 3 } });
    t.Insertar(Computador { "c4", { 4 } });
    t.Insertar(Computador { "c5", { 5 } });

    std::cout << "Todo insertado" << std::endl;

    // Comprobamos que da la vuelta bien.
    std::cout << std::endl;
    std::cout << "Dando una vuelta " << std::endl;
    for(int i = 0; i < 6; i++)
    {
        std::cout << "El ordenador con token es el ordenador " << t.TieneToken().nombre << std::endl;
        if(i != 5) t.PasarToken();
    }

    std::cout << std::endl;
    std::cout << "Viendo las tramas" << std::endl;
    for(int i = 0; i < 5; i++)
    {
        std::cout << "El ordenador " << t.TieneToken().nombre << " tiene trama " << t.TieneToken().tr << std::endl;
        t.PasarToken();
    }

    // Le mando la trama del 3 al 1 y me aseguro que se manda correctamente.
    std::cout << std::endl;
    std::cout << "Probando \"Enviar Trama\" " << std::endl;
    t.EnviarTrama("c1", "c5");
    for(int i = 0; i < 5; i++)
    {
        std::cout << "El ordenador " << t.TieneToken().nombre << " tiene trama " << t.TieneToken().tr << std::endl;
        t.PasarToken();
    }

    std::cout << std::endl;
    std::cout << "Probando \"Eliminar\" " << std::endl;
    t.Eliminar();
    for(int i = 0; i < 5; i++)
    {
        std::cout << "El ordenador con token es el ordenador " << t.TieneToken().nombre << std::endl;
        if(i != 4) t.PasarToken();
    }

    std::cout << std::endl;
    std::cout << "Tests OK :) \n";

    return 0;
}
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

Postfijo

Podemos expresar una expresión aritmética de dos formas diferentes:

- Infijo: $(A+B)*(C+D)$ | | $X/Y*Z+W$
- Postfijo: $AB+CD+*$ | | $XY/Z*W+$

Dada una expresión postfijo en forma de cadena de caracteres y una lista de las variables que aparecen en dicha expresión junto a sus respectivos valores, programa una función que devuelva el resultado de evaluar la expresión, haciendo uso del TAD Pila.

Para simplificar el problema asumiremos que la expresión postfijo es correcta, que consta sólo de dos operadores elementales (+, -, * y /) y que los operandos son variables cuyo nombre constan de un solo carácter.

Ejemplo: Teniendo las variables A=2, B=3, C=1, D=2 $AB+CD+*= 15$

```
#include <iostream>
#include "pilaenla.hpp"
#include <string>
using namespace std;

struct Variable
{
    char nombre;    //Nombre
    int valor;

    Variable(char n = ' ', int i = 0) : nombre(n), valor(i) {}
};

int buscarIndice(const Variable variables[], char nombre) {
    for (int i = 0; variables[i].nombre != '\0'; ++i) {
        if (variables[i].nombre == nombre) {
            return i; // Retorna el índice si se encuentra la variable
        }
    }
    return -1; // Retorna -1 si no se encuentra la variable
}
```

```
int evaluarExpresionPostfijo(const string& expresion, const Variable variables[])
{
    Pila<int> pila;

    for (char c : expresion)
    {
        if (isalpha(c)) // Si es una variable
        {
            //int indice = c-'A'; // Convertir carácter a índice (A=0, B=1,...) - Solo funciona ordenado.
            int indice = buscarIndice(variables, c); // Obtener el índice buscándolo
            if (indice != -1)
                pila.push(variables[indice].valor); // Empujar el valor de la variable a la pila
            else
                throw invalid_argument("ERROR: Valor no encontrado en la lista...");
        }
        else if (c == '+' || c == '-' || c == '*' || c == '/') // Si es un operador
        {
            int operand2 = pila.tope(); pila.pop();
            int operand1 = pila.tope(); pila.pop();

            int result = 0;
            switch(c)
            {
                case '+':      result = operand1 + operand2;
                               break;
                case '-':      result = operand1 - operand2;
                               break;
                case '*':      result = operand1 * operand2;
                               break;
                case '/':      result = operand1 / operand2;
                               break;
                default:       break;
            }

            pila.push(result);
        }
    }

    return pila.tope();
}

int main()
{
    //((A+B) * (C+D)) = 15
    string expresion = {"AB+CD*"};
    //((A+B) * (C+D)) * A-E = -60
    //string expresion = {"AB+CD+*AE-*"};
    //((A+B) * (C+D)) * B-E = -45
    //string expresion = {"AB+CD+*BE-*"};

    Variable variables[] = {
        Variable('A', 2),
        Variable('C', 1),
        Variable('B', 3),
        Variable('D', 2),
        Variable('E', 6),
    };

    std::cout << "El valor del postfijo es de " << evaluarExpresionPostfijo(expresion, variables) << "." << std::endl;

    return 0;
}
```

Cartelera

Se desea un TAD para carteleras de espectáculos, que representen un sistema informático con información relativa a los espectáculos ofrecidos al público por una serie de salas. Cada sala puede ofrecer varios espectáculos. El TAD debe contener las siguientes operaciones:

- Crear una cartelera vacía.
- Añadir una sala a la cartelera (esta no se modifica si la sala ya existía).
- Añadir un espectáculo a la oferta de una sala ya existente (el sistema no se modifica si la sala ya ofrecía ese espectáculo).
- Eliminar una sala, junto con sus espectáculos (el sistema no se modifica si la sala no existía),
- Eliminar un espectáculo de una sala (el sistema no se modifica si la sala no existía o si existía pero no ofrecía ese espectáculo).

```
#ifndef CARTELERA_HPP
#define CARTELERA_HPP
#include <iostream>
#include <cassert>
#include <cstdio>
#include <cstring>
#include <vector>
#include "listaenla.hpp"
using namespace std;

//Tipos
typedef int Espectaculo;
struct Sala
{
    size_t id_sala;
    Lista<Espectaculo> espectaculos; // Constructor por defecto de Sala
    Sala(size_t id = 0) : id_sala { id } { }
};

class Cartelera{
public:
    Cartelera() = default; //Constructor
    void InsertarSala(int sala); //id de la sala
    void InsertarEspectaculo(Espectaculo espectaculo, int sala); //id de la sala
    void EliminarSala(int sala); //id de la sala
    void EliminarEspectaculo(Espectaculo espectaculo, int sala); //id del espectaculo, id de la sala

    //Observadores
    const Lista<Sala>& MostrarSalas() const noexcept;
    Lista<Espectaculo>& MostrarEspectaculos(int sala);

    ~Cartelera() = default; //Destructor

private:
    Lista<Sala> Salas;
    Lista<Sala>::posicion BuscarSala(int sala);
    Lista<Espectaculo>::posicion BuscarEspectaculo(Espectaculo espectaculo, Lista<Sala>::posicion sala);
};

//<--SALAS-->
Lista<Sala>::posicion Cartelera::BuscarSala(int sala)
{
    Lista<Sala>::posicion pos = Salas.primer();
    bool encontrado = false;

    while(pos != Salas.fin() && !encontrado)
    {
        if(Salas.elemento(pos).id_sala == sala)
            encontrado = true;
        else
            pos = Salas.siguiente(pos);
    }

    return pos;
}
```

SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
void Cartelera::InsertarSala(int sala)
{
    Lista<Sala>::posicion pos = BuscarSala(sala);
    //Si no se encuentra en la lista Salas, se inserta
    if(pos == Salas.fin())
        Salas.insertar(Sala(sala), Salas.fin());
}

void Cartelera::EliminarSala(int sala)
{
    Lista<Sala>::posicion pos = BuscarSala(sala);

    if(pos != Salas.fin())
        Salas.eliminar(pos);
}

//<--ESPECTACULOS-->

Lista<Espectaculo>::posicion Cartelera::BuscarEspectaculo(Espectaculo espectaculo, Lista<Sala>::posicion sala)
{
    Lista<Espectaculo>::posicion posE = Salas.elemento(sala).espectaculos.primer();
    bool encontrado = false;

    while(posE != Salas.elemento(sala).espectaculos.fin() && !encontrado)
    {
        if(Salas.elemento(sala).espectaculos.elemento(posE) == espectaculo)
            encontrado = true;
        else
            posE = Salas.elemento(sala).espectaculos.siguiente(posE);
    }

    return posE;
}

void Cartelera::InsertarEspectaculo(Espectaculo espectaculo, int sala)
{
    Lista<Sala>::posicion posS = BuscarSala(sala);
    assert(posS != Salas.fin());

    if(BuscarEspectaculo(espectaculo, posS) == Salas.elemento(posS).espectaculos.fin())

        Salas.elemento(posS).espectaculos.insertar(Espectaculo(espectaculo), Salas.elemento(posS).espectaculos.fin());
}

void Cartelera::EliminarEspectaculo(Espectaculo espectaculo, int sala)
{
    Lista<Sala>::posicion posS = BuscarSala(sala);
    if(posS != Salas.fin())
    {
        Lista<Espectaculo>::posicion posE = BuscarEspectaculo(espectaculo, posS);
        if(posE != Salas.elemento(posS).espectaculos.fin())
            Salas.elemento(posS).espectaculos.eliminar(posE);
    }
}

const Lista<Sala>& Cartelera::MostrarSalas() const noexcept
{
    return Salas;
}

Lista<Espectaculo>& Cartelera::MostrarEspectaculos(int sala)
{
    Lista<Sala>::posicion posS = BuscarSala(sala);
    assert(posS != Salas.fin());

    return Salas.elemento(posS).espectaculos;
}

#endif //CARTELERA_HPP

////////////////////////////////////
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
#include "cartelera.hpp"

void PrintLista(Cartelera c)
{
    Lista<Sala> S = c.MostrarSalas();
    auto posS = S.primer();

    while (posS != S.fin())
    {
        std::cout << "Sala --> " << S.elemento(posS).id_sala << std::endl;

        Lista<Espectaculo> esps = c.MostrarEspectaculos(S.elemento(posS).id_sala);
        auto posE = esps.primer();

        while (posE != esps.fin())
        {
            std::cout << " Espectaculo --> " << S.elemento(posS).espectaculos.elemento(posE) << std::endl;
            posE = esps.siguiete(posE);
        }

        posS = S.siguiete(posS);
    }
    std::cout << std::endl;
}

int main()
{
    Cartelera c;

    std::cout << "<--Insertar Salas-->" << std::endl << std::endl;
    c.InsertarSala(1);
    c.InsertarSala(2);
    c.InsertarSala(3);
    c.InsertarSala(4);
    c.InsertarSala(5);

    PrintLista(c);

    std::cout << "<--Eliminar Salas 1 y 3-->" << std::endl << std::endl;
    c.EliminarSala(1);
    c.EliminarSala(3);

    PrintLista(c);

    std::cout << "<--Insertar Espectaculos-->" << std::endl << std::endl;
    c.InsertarEspectaculo(Espectaculo(10),5);
    c.InsertarEspectaculo(Espectaculo(10),2);
    c.InsertarEspectaculo(Espectaculo(60),2);
    c.InsertarEspectaculo(Espectaculo(90),4);
    c.InsertarEspectaculo(Espectaculo(43),4);

    PrintLista(c);

    std::cout << "<--Eliminar Espectaculo 90 en 4-->" << std::endl << std::endl;
    c.EliminarEspectaculo(90,4);

    PrintLista(c);

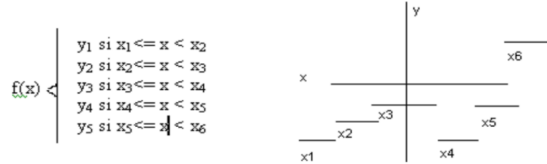
    std::cout << "<--Nada deberia cambiar-->" << std::endl << std::endl;
    c.InsertarSala(5); //Ya existe la sala
    c.EliminarSala(6); //No existe la sala
    c.InsertarEspectaculo(43, 4); //Ya existe el espectaculo
    c.EliminarEspectaculo(41, 4); //No existe el espectaculo

    PrintLista(c);

    return 0;
}
```

Escalonada

Una función $f: \mathbb{R} \rightarrow \mathbb{R}$ es escalonada cuando consiste en una sucesión de funciones constantes definidas en subintervalos disjuntos y contiguos, es decir, f puede definirse mediante condiciones de la forma $f(x)=y_i$, si $x_i \leq x < x_{i+1}$, donde los valores y_i son distintos para subintervalos adyacentes. Cada uno de los puntos (x_i, y_i) en los que la función cambia de valor se llama salto o escalón.



(40%) Especificar un TAD para las funciones escalonadas con un número finito de saltos, que incluya las siguientes operaciones:

- Crear una función constante $f(x)=y_i$, definida a partir de x_i
- Añadir un nuevo salto en el punto (x,y) a una función. Si ya existe un salto en las coordenadas x , se sustituye por el nuevo.
- Eliminar el escalón definido en el subintervalo al que pertenece la coordenada x . El escalón anterior al eliminado se prolonga hasta el siguiente.
- Calcular el valor de una función en un punto dado.
- Calcular el valor mínimo de una función escalonada
- Calcular el valor máximo de una función escalonada
- Hacer una traslación de una función w unidades horizontales y z unidades verticales, siendo w y z números reales.
- Destruir una función

```

/*****
OPERACIONES
Escalonada(const salto& s)
    Postcondición: Se crea una función constante f(x) = yi definida a partir de xi.
void insertarSalto(const salto& s)
    Postcondición: Se inserta un nuevo salto. Si ya existía un salto en la misma
                    posición, se sustituye por el nuevo.
void eliminarSalto(const salto& s)
    Precondición: Existe un salto en la posición xi.
    Postcondición: Se elimina el salto.
double valorFuncion(double x) const
    Postcondición: Devuelve el valor de la función en el punto dado.
double valorMinimo() const
    Postcondición: Se devuelve el valor mínimo de la función escalonada.
double valorMaximo() const
    Postcondición: Se devuelve el valor máximo de la función escalonada.
void traslacion(double w, double z)
    Precondición: w y z -> R
    Postcondición: Se hace una traslación de la función w unidades horizontales
                    y z unidades verticales
*****/

#ifndef ESCALONADA_HPP
#define ESCALONADA_HPP
#include <iostream>
#include "listaenla.hpp"
#include <cassert>

```

```
struct salto
{
    double xi , yi ;
    salto(double a = 0, double b = 0) : xi(a), yi(b) {};
};

class Escalonada{
public:
    explicit Escalonada(const salto& s);
    void insertarSalto(const salto& s);
    void eliminarSalto (const salto& s);
    double valorFuncion(double x) const;
    double valorMinimo() const;
    double valorMaximo() const;
    void translacion ( double w , double z);
    ~Escalonada() = default;
private:
    double valor_constante;
    Lista<salto> funcion;
    typedef typename Lista<salto>::posicion posicion;
};

Escalonada::Escalonada(const salto& s): valor_constante { s.yi }
{
    funcion.insertar(s, funcion.fin());
}

void Escalonada::insertarSalto(const salto& s)
{
    posicion pos = funcion.primer();
    bool insertado = false;

    while (!(pos == funcion.fin()) && !insertado)
    {
        if (funcion.elemento(pos).xi == s.xi)    //Si encuentra una funcion con la misma x.
        {
            funcion.elemento(pos).yi = s.yi; //Se cambia su yi
            insertado = true;
            //std::cout << "Insertado de forma exitosa!" << std::endl;
        }
        else
            pos = funcion.siguiente(pos);
    }

    if (!insertado)                //Si la x no se encuentra en la funcion
    {
        funcion.insertar(s, funcion.fin());    //Se inserta el salto nuevo.
        //std::cout << "Insertado de forma exitosa!" << std::endl;
    }
}

void Escalonada::eliminarSalto(const salto& s)
{
    posicion pos = funcion.primer();
    bool encontrado = false;

    while (!(pos == funcion.fin()) && !encontrado)
    {
        if (funcion.elemento(pos).xi == s.xi)    //Si encuentra un salto en xi
        {
            funcion.eliminar(pos);                //Lo elimina
            encontrado = true;
            //std::cout << "Eliminado de forma exitosa!" << std::endl;
        }
        else
            pos = funcion.siguiente(pos);
    }
}
```


SABEMOS QUE PUEDES DESCARGARTE UN TRABAJO COMPLETO DE CUALQUIER ESCRITOR,
PERO ZAMBULLIRTE EN SU OBRA Y NO PARAR DE LEER HASTA QUE SE TE HAGA DE DÍA...

ESO, SE MERECE UN AQUARIUS.

AQUARIUS es una marca registrada de The Coca-Cola Company.



Kuki MW
PRÁCTICAS AAED

```
double Escalonada::valorFuncion(double x) const
{
    posicion pos = funcion.primer();

    while (pos != funcion.fin())
    {
        if((funcion.siguiete(pos) == funcion.fin()) && (x >= funcion.elemento(pos).xi)
            || (funcion.elemento(pos).xi <= x) && (funcion.elemento(funcion.siguiete(pos)).xi > x))
        {
            return funcion.elemento(pos).yi;
        }
        else
        {
            pos = funcion.siguiete(pos);
        }
    }
    // Nos dan un valor de x que está antes que cualquier salto, por lo tanto es el valor constante
    return valor_constante;
}

void Escalonada::translacion(double w, double z)
{
    posicion pos = funcion.primer();

    while (pos != funcion.fin())
    {
        funcion.elemento(pos).xi += w;
        funcion.elemento(pos).yi += z;
        pos = funcion.siguiete(pos);
    }
    valor_constante += z;
}

double Escalonada::valorMinimo() const
{
    posicion pos = funcion.primer();
    double minimo = valor_constante;

    if (pos != funcion.fin())
    {
        minimo = funcion.elemento(pos).yi;

        while (pos != funcion.fin())
        {
            double auxy = funcion.elemento(pos).yi;
            if (auxy < minimo)
                minimo = auxy;
            pos = funcion.siguiete(pos);
        }
    }
    // Si no hay ningún salto, devolvemos el valor constante.
    return minimo;
}

double Escalonada::valorMaximo() const
{
    posicion pos = funcion.primer();
    double maximo = valor_constante;

    if (pos != funcion.fin())
    {
        maximo = funcion.elemento(pos).yi;

        while (pos != funcion.fin())
        {
            double auxy = funcion.elemento(pos).yi;
            if (auxy > maximo)
                maximo = auxy;
            pos = funcion.siguiete(pos);
        }
    }
    // Si no hay ningún salto, devolvemos el valor constante.
    return maximo;
}

#endif // "ESCALONADA_HPP"
```

AQUARIUS
SUDAR
ES BELLO



WUOLAH

```
int main()
{
    Escalonada c (salto(1,2));

    std::cout << std::endl<< "<-- INSERTAR SALTOS -->"<< std::endl<< std::endl;
    c.insertarSalto(salto(3, 6));
    c.insertarSalto(salto(5, 3));
    c.insertarSalto(salto(7, 6));

    std::cout << std::endl << "<-- VALOR EN FUNCION DE X -->"<< std::endl<< std::endl;
    std::cout << "Valor en -1: " << c.valorFuncion(-1) << std::endl; //2
    std::cout << "Valor en 1: " << c.valorFuncion(1) << std::endl; //2
    std::cout << "Valor en 2: " << c.valorFuncion(2) << std::endl; //2
    std::cout << "Valor en 3: " << c.valorFuncion(3) << std::endl; //6
    std::cout << "Valor en 4: " << c.valorFuncion(4) << std::endl; //6
    std::cout << "Valor en 5: " << c.valorFuncion(5) << std::endl; //3
    std::cout << "Valor en 6: " << c.valorFuncion(6) << std::endl; //3
    std::cout << "Valor en 7: " << c.valorFuncion(7) << std::endl; //6
    std::cout << "Valor en 8: " << c.valorFuncion(8) << std::endl; //6

    std::cout << std::endl<< "<-- MINIMO \"Y\" Y MAXIMO \"Y\" -->"<< std::endl<< std::endl;
    std::cout << "Min:" << c.valorMinimo() << std::endl; // assert(c.valorMinimo() == 2);
    std::cout << "Max:" << c.valorMaximo() << std::endl; // assert(c.valorMaximo() == 6);

    std::cout << std::endl<< "<-- VALOR EN FUNCION DE X TRAS TRASLACION (2,3)-->"<< std::endl<< std::endl;
    c.translacion(2, 3);
    // salto(1,2) -> 1+2,2+3 -> salto(3,5)

    std::cout << "Valor en -1: " << c.valorFuncion(-1) << std::endl; //5
    std::cout << "Valor en 1: " << c.valorFuncion(1) << std::endl;
    std::cout << "Valor en 2: " << c.valorFuncion(2) << std::endl;
    std::cout << "Valor en 3: " << c.valorFuncion(3) << std::endl; //9
    std::cout << "Valor en 4: " << c.valorFuncion(4) << std::endl;
    std::cout << "Valor en 5: " << c.valorFuncion(5) << std::endl; //6
    std::cout << "Valor en 6: " << c.valorFuncion(6) << std::endl;
    std::cout << "Valor en 7: " << c.valorFuncion(7) << std::endl; //9
    std::cout << "Valor en 8: " << c.valorFuncion(8) << std::endl;
    std::cout << "Valor en 9: " << c.valorFuncion(9) << std::endl;

    std::cout << std::endl<< "<-- MINIMO \"Y\" Y MAXIMO \"Y\" -->"<< std::endl<< std::endl;
    std::cout << "Min:" << c.valorMinimo() << std::endl; // assert(c.valorMinimo() == 5);
    std::cout << "Max:" << c.valorMaximo() << std::endl; // assert(c.valorMaximo() == 9);

    std::cout << std::endl<< "<-- ELIMINAR SALTOS -->"<< std::endl<< std::endl;
    c.eliminarSalto(salto(3, 5));
    std::cout << "Tras eliminar (3,5), Valor en 4: " << c.valorFuncion(4) << std::endl;
    //assert(c.valorFuncion(4));

    c.eliminarSalto(salto(5, 9));
    std::cout << "Tras eliminar (5,9), Valor en 6: " << c.valorFuncion(6) << std::endl;

    std::cout << std::endl << "<-- MINIMO \"Y\" Y MAXIMO \"Y\" -->"<< std::endl<< std::endl;
    std::cout << "Min:" << c.valorMinimo() << std::endl; // assert(c.valorMinimo() == 5);
    std::cout << "Max:" << c.valorMaximo() << std::endl; // assert(c.valorMaximo() == 5);

    std::cout << std::endl << "<----->" << std::endl<< std::endl;
    std::cout << "Todo OK :)\n";
    std::cout << std::endl << "<----->" << std::endl<< std::endl;

    return 0;
}
```